01-485/TTL Communicate SDK

Complete documentation collection

English edition

1.1 SDK Download

1.2 C API Interface Description

1.3 C++ API Interface Description

1.4 ROS & Python API Interface Description

19 October 2026

A consolidated reference for the C, C++, ROS and Python interfaces. Tables, code examples and SDK file listings are presented in English with consistent formatting and page numbering.

The SDK Download chapter provides the SDK repository link.

Contents

1.1 SDK Download **4**

1.2 C API Interface Description **5**

1. Introduction 5

2. Function Introduction 5

3. Instructions for Use 5

4. Explanation of API Part Macro Definitions 6

4.1. Communication Status Code Description 6

4.2. Protocol Instruction Macro Description 7

4.3. Device ID Macro Definition 7

5. API Interface Function Description 8

5.1. PortHandler 8

5.2. PacketHandler 12

5.3. groupSyncWrite.h 23

5.4. GroupSyncRead 25

5.5. GroupBulkWrite 27

5.6. GroupBulkRead 28

1.3 C++ API Interface Description **31**

1. Introduction 31

2. Function Introduction 31

3. Instructions for Use 31

4. Explanation of API Part Macro Definitions 33

4.1. Communication Status Code Description 33

4.2. Protocol Instruction Macro Description 33

4.3. Device ID Macro Definition 33

5. API Interface Function Description 34

5.1. PortHandler . . . 34
5.2. PacketHandler . . . 37
5.3. GroupSyncWrite . . . 47
5.4. GroupSyncRead . . . 49
5.5. GroupBulkWrite . . . 52
5.6. GroupBulkRead . . . 54

1.4 ROS & Python API Interface Description 58

1. Introduction . . . 58
2. Function Introduction . . . 58
3. Instructions for Use . . . 58
4. Explanation of API Part Macro Definitions . . . 60
4.1. Communication Status Code Description . . . 60
4.2. Protocol Instruction Macro Description . . . 60
4.3. Device ID Macro Definition . . . 60
5. API Interface Function Description . . . 61
5.1. PortHandler . . . 61
5.2. PacketHandler . . . 64
5.3. GroupSyncWrite . . . 76
5.4. GroupSyncRead . . . 78
5.5. GroupBulkWrite . . . 81
5.6. GroupBulkRead . . . 83

1.1 SDK Download

SDK Download

1.2 C API Interface Description

1. Introduction

This document is a description of the C secondary development interface

Supported platforms are: Linux, Windows, macOS

2. Function Introduction

The C API implements the functions of serial port data reception and transmission, as well as data encapsulation and parsing for SIAR Protocol 1.0 & 2.0, facilitating users to quickly implement communication with SIAR series motors using interface functions. It can be used in Windows, Linux, and macOS environments.

3. Instructions for Use

The contents of the C API folder are as shown in the figure below, providing three parts: implementation source code, examples, and compiled DLL library files. Among them, the include and src folders contain the implementation source code, the example folder provides examples, and the build folder is the path to the DLL library files generated for each platform.

Name	Modified	Type	Size
build	2024/11/19 14:07	Folder	
example	2024/11/19 14:07	Folder	
include	2024/11/19 14:07	Folder	
src	2024/11/19 14:07	Folder	

You can call the C API using one of the following two methods:

(1) Copy the implementation source code to the project

(2) Copy the compiled dll library files to the project and add references to the dll libraries

The general steps for using this library are as follows

1. Import Header File

```
#include "siardk.h"                                  // Uses siar SDK library
```

2. Call the portHandler function to obtain the device number, and call the packetHandle function to initialize the PacketHanlder structure, which is responsible for storing protocol data.

```
#define DEVICENAME                "/COM5"
// Protocol version
#define PROTOCOL_VERSION            2.0
// Initialize PortHandler Structs
// Set the port path
// Get methods and members of PortHandlerLinux or PortHandlerWindows
int port_num = portHandler(DEVICENAME);

// Initialize PacketHandler Structs
packetHandler();
```

3. Set the baud rate and open the serial communication device

```
// Open port
if (openPort(port_num))
{
   printf("Succeeded to open the port!\n");
}
```

```
else
{
   printf("Failed to open the port!\n");
   printf("Press any key to terminate...\n");
   getch();
   return 0;
 }

 // Set port baudrate
 if (setBaudRate(port_num, BAUDRATE))
 {
    printf("Succeeded to change the baudrate!\n");
 }
 else
 {
    printf("Failed to change the baudrate!\n");
    printf("Press any key to terminate...\n");
    getch();
    return 0;
  }
```

4. Use relevant function interfaces to perform data encapsulation, parsing, reception, and transmission functions. For example, the code for pinging an online device is as follows.

```
// Try to ping the siar
// Get siar model number
siar_model_number = pingGetModelNum(port_num, PROTOCOL_VERSION, SIAR_ID);
if ((siar_comm_result = getLastTxRxResult(port_num, PROTOCOL_VERSION)) != COMM_SUCCESS)
{
   printf("%s\n", getTxRxResult(PROTOCOL_VERSION, siar_comm_result));
}
else if ((siar_error = getLastRxPacketError(port_num, PROTOCOL_VERSION)) != 0)
{
   printf("%s\n", getRxPacketError(PROTOCOL_VERSION, siar_error));
}
else
{
  printf("[ID:%03d] ping Succeeded. siar model number : %d\n", SIAR_ID, siar_model_number);
}
```

5. Finally, close the serial communication

```
 // Close port
closePort(port_num);
```

4. Explanation of API Part Macro Definitions

4.1. Communication Status Code Description

Value	Macro Definition	Status Resolution
0	COMM_SUCCESS	Communication success status
-1000	COMM_PORT_BUSY	Port Usage Status
-1001	COMM_TX_FAIL	Status of failed data packet transmission
-1002	COMM_RX_FAIL	Status of failed packet reception
-2000	COMM_TX_ERROR	Transmission Packet Error Status
-3000	COMM_RX_WAITING	Receiving Packet Waiting State
-3001	COMM_RX_TIMEOUT	Receive Packet Timeout Status
-3002	COMM_RX_CORRUPT	Status of corrupted received data packets
-9000	COMM_NOT_AVAILABLE	Status unavailable in Protocol 1.0

4.2. Protocol Instruction Macro Description

Value	Macro Definition	Instruction Parsing
1	INST_PING	ping command value
2	INST_READ	Instruction value of Read
3	INST_WRITE	Written instruction value
4	INST_REG_WRITE	Instruction value written to the register
5	INST_ACTION	Instruction value of the operation
6	INST_FACTORY_RESET	Command value for restoring factory settings
131	INST_SYNC_WRITE	Command value for synchronous write
146	INST_BULK_READ	Instruction values read in batch
8	INST_REBOOT	Instruction value of Reboot
16	INST_CLEAR	Clear command value
85	INST_STATUS	Instruction value of the state
130	INST_SYNC_READ	Instruction value for synchronous read
138	INST_FAST_SYNC_READ	Instruction value for fast synchronous read
147	INST_BULK_WRITE	Instruction value for batch writing

4.3. Device ID Macro Definition

Value	Macro Definition	ID Parsing
0xFE	BROADCAST_ID	Broadcast ID
0xFC	MAX_ID	Maximum ID Value

5. API Interface Function Description

5.1. PortHandler

Method	Description
portHandler()	Obtain the device port number under the corresponding hardware platform
openPort()	Open Serial Port
closePort()	Close Serial Port
clearPort()	Refresh Serial Port
setPortName()	Set Device Name
getPortName()	Get Device Name
setBaudrate()	Set Baud Rate
getBaudrate()	Get Baud Rate
getBytesAvailable()	Check the number of bytes that can be read in the port buffer (Linux ports only)
readPort()	Read bytes from the port buffer
writePort()	Write ByteDance to the port buffer
setPacketTimeout()	Set timeout
setPacketTimeoutMSec	Get the time since the start
isPacketTimeout()	Check if communication is successful when timeout occurs

5.1.1. portHandler

int portHandler(const char *port_name)	
Functionality	Obtain the device port number under the corresponding hardware platform
Parameter	port_name: used to pass in a serial port name
Return Value	None

5.1.2. openPort

Uint8_t openPort(int port_num)	
Functionality	Open Serial Port
Parameter	port_num: Port Number
Return Value	When the port is successfully opened, this function will return true; otherwise, it will return false

5.1.3. closePort

void closePort(int port_num)	
Functionality	Close Serial Port
Parameter	port_num: Port Number
Return Value	None

5.1.4. clearPort

void clearPort(int port_num)	
Functionality	Clear Serial Port
Parameter	port_num: Port Number
Return Value	None

5.1.5. setPortName

void setPortName(int port_num, const char *port_name)	
Functionality	Set Port Name
Parameter	1. port_num: Pass in a device Port Number 2. port_name: Pass in a serial port name
Return Value	None

5.1.6. getPortName

char* getPortName(int port_num)	
Functionality	Get Port Name
Parameter	port_num: Port Number
Return Value	Returns a string pointer to the port name

5.1.7. setBaudrate

uint 8_t setBaudrate(int port_num,const int baudrate)	
Functionality	Set Baud Rate
Parameter	1. port_num: Port Number 2. baudrate: Target Baud Rate
Return Value	Returning true indicates that the baud rate was successfully set, while returning false indicates that the baud rate setting failed.

5.1.8. getBaudrate

int getBaudrate(int port_num)	
Functionality	Get the baud rate of the current serial port communication
Parameter	port_num: Port Number
Return Value	Returns the currently set baud rate

5.1.9. getBytesAvailable

int getBytesAvailable(int port_num)	
Functionality	Get the number of currently available bytes
Parameter	port_num: Port Number
Return Value	Returns the number of currently available bytes

5.1.10. readPort

int readPort(int port_num, uint8_t *packet, int length)	
Functionality	Read data from the port into the specified buffer
Parameter	1. port_num: Port Number 2. packet: The data packet to be read currently 3. length: the length of bytes to be read
Return Value	Returns the number of bytes actually read

5.1.11. writePort

int writePort(int port_num, uint8_t *packet, int length)	
Functionality	Write the data packet to the specified port
Parameter	1. port_num: Port Number 2. packet: Pointer to the packet to be written 3. length: pointer to the length of the data packet
Return Value	A result of 0 for the operation indicates success, while a non-zero result indicates an error

5.1.12. setPacketTimeout

void setPacketTimeout(int port_num, uint16_t packet_length)	
Functionality	Set the timeout period for the data packet
Parameter	1. port_num: Port Number 2. packet_length: A pointer to the length of the packet for which the timeout is to be set
Return Value	None

5.1.13. setPacketTimeoutMSec

void setPacketTimeoutMSec(int port_num, double msec)	
Functionality	Set timeout using millisecond value
Parameter	1. port_num: Port Number 2. msec: millisecond
Return Value	Returning true indicates a timeout, while false indicates no timeout

5.1.14. isPacketTimeout

uint8_t isPacketTimeout(int port_num)	
Functionality	Determine whether the data packet has timed out
Parameter	port_num: Port Number
Return Value	Returning true indicates a timeout, while false indicates no timeout

5.2. PacketHandler

Method	Description
PacketHandler	Initialize the members of the data pointer structure of the data packet
getTxRxResult	Get the results of send and receive operations
getRxPacketError	Get the corresponding error message string
getLastTxRxResult	Get the result code of the latest send or receive operation under the specified port number and protocol version
getLastRxPacketError	Get the error information of the latest received data packet
setDataWrite	Set the data write operation under the specified port number and protocol version
getDataRead	Read data under the specified port number and protocol version
txPacket	Send packets under the specified port number and protocol version
rxPacket	Receive packets under the specified port number and protocol version
txRxPacket	Simultaneously send and receive data packets under the specified port number and protocol version
ping	Send a data packet to the specified port number and protocol version to test network connectivity
pingGetModelNum	Get the number of modes under the specified port number and protocol version
broadcastPing	Broadcast a data packet to the specified port number and protocol version
getBroadcastPingResult	Get the broadcast packet results for the specified port number and protocol version
reboot	Restart the device with the specified port number and protocol version
clearMultiTurn	Clear multi-round conversations for the specified port and protocol version
factoryReset	Perform a factory reset on devices with the specified port and protocol version
readTx	Read transmission data from a device with a specified port and protocol version
readRx	Read received data from a device with a specified port and protocol version
readTxRx	Simultaneously read transmit and receive data from devices with specified ports and protocol versions
read1ByteTx	Send a 1-ByteDance read command packet
read1ByteRx	Receive 1 ByteDance read status packet
read1ByteTxRx	Send and receive 1 ByteDance packet
read2ByteTx	Send a 2-ByteDance read command packet
read2ByteRx	Receive a 2-ByteDance read status data packet
read2ByteTxRx	Send and receive 2 ByteDance packets
void read4ByteTx	Send a 4-ByteDance read instruction packet
read4ByteRx	Receive a 4-ByteDance read status data packet
read4ByteTxRx	Send and receive 4 ByteDance packets
writeTxOnly	Send N ByteDance write command data packets
writeTxRx	Send and receive N ByteDance packets
write1ByteTxOnly	Send a 1 ByteDance write command data packet

Method	Description
write1ByteTxRx	Send and receive 1 ByteDance packet
write2ByteTxOnly	Send a 2-ByteDance write command data packet
write2ByteTxRx	Send and receive 2 ByteDance packets
write4ByteTxOnly	Send a 4-ByteDance write command data packet
write4ByteTxRx	Send and receive 4 ByteDance packets
regWriteTxOnly	Send register write instruction data packet
regWriteTxRx	Send and receive register write packets
syncReadTx	Send N ByteDance synchronous read command data packet
syncWriteTxOnly	Send N ByteDance synchronous write command data packet
bulkReadTx	Send N ByteDance batch read command data packet
bulkWriteTxOnly	Send N ByteDance batch write command data packet

5.2.1. PacketHandler

void packetHandler()	
Functionality	Initialize the members of the data pointer structure of the data packet
Parameter	None
Return Value	None

5.2.2. getTxRxResult

const char *getTxRxResult(int protocol_version, int result)	
Functionality	Get the results of send and receive operations
Parameter	1. protocol_version: The version number of the protocol 2. result: Result code, indicating the result status or type of the operation
Return Value	Returns a pointer to the result information string

5.2.3. getRxPacketError

const char *getRxPacketError(int protocol_version, uint8_t error)	
Functionality	Get the corresponding error message string
Parameter	1. protocol_version: Protocol Version Number 2. error: Error Code
Return Value	Returns a pointer to the error message string

5.2.4. getLastTxRxResult

int getLastTxRxResult(int port_num, int protocol_version)	
Functionality	Get the result code of the latest send or receive operation under the specified port number and protocol version
Parameter	1. port_num: Specified Port Number 2. protocol_version: Specified protocol version number
Return Value	Returns the result code of the most recent transmission or reception operation

5.2.5. getLastRxPacketError

uint8_t getLastRxPacketError(int port_num, int protocol_version)	
Functionality	Get error information about the most recently received packet
Parameter	1. port_num: Specified Port Number 2. protocol_version: Protocol Version
Return Value	Return Error Code Pointer

5.2.6. setDataWrite

void setDataWrite(int port_num, int protocol_version, uint16_t data_length, uint16_t data_pos, uint32_t data)	
Functionality	Set the data write operation under the specified port number and protocol version
Parameter	1. port_num: Specified Port Number 2. protocol_version: Specified protocol version number 3. data_length: The length of the written data, with optional values of 1, 2, or 4 4. data_pos: Data Position or Offset 5. data: Pointer to the data to be written
Return Value	None

5.2.7. getDataRead

uint32_t getDataRead(int port_num, int protocol_version, uint16_t data_length, uint16_t data_pos)	
Functionality	Read data under the specified port number and protocol version
Parameter	1. port_num: Specified Port Number 2. protocol_version: Specified protocol version number 3. data_length: The length of the written data, with optional values of 1, 2, or 4 4. data_pos: Data Position or Offset
Return Value	Returns a pointer to the read data

5.2.8. txPacket

void txPacket(int port_num, int protocol_version)	
Functionality	Send packets under the specified port number and protocol version
Parameter	1. port_num: A pointer to the Port Number to be operated on 2. protocol_version: Pointer to the protocol version number to be operated on
Return Value	None

5.2.9. rxPacket

void rxPacket (int port_num, int protocol_version)	
Functionality	Receive packets under the specified port number and protocol version
Parameter	1. port_num: Specified Port Number 2. protocol_version: Specified protocol version number
Return Value	Returns a pointer to the received data

5.2.10. txRxPacket

void txRxPacket(int port_num, int protocol_version)	
Functionality	Simultaneously send and receive data packets under the specified port number and protocol version
Parameter	1. port_num: Specified Port Number 2. protocol_version: Specified protocol version number
Return Value	None

5.2.11. ping

void ping（int port_num, int protocol_version, uint8_t id）	
Functionality	Send a data packet to the specified port number and protocol version to test network connectivity
Parameter	1. port_num: A pointer to the Port Number to be operated on 2. protocol_version: Pointer to the protocol version number to be operated on 3. id: Pointer to the packet identifier
Return Value	None

5.2.12. pingGetModelNum

uint16_t pingGetModelNum(int port_num, int protocol_version, uint8_t id)	
Functionality	Get the number of modes under the specified port number and protocol version
Parameter	1. port_num: Specified Port Number 2. protocol_version: Specified protocol version number 3. id: Device ID
Return Value	Return the number of modes

5.2.13. broadcastPing

void broadcastPing(int port_num, int protocol_version)	
Functionality	Broadcast a data packet to the specified port number and protocol version
Parameter	1. port_num: A pointer to the Port Number to be operated on 2. protocol_version: Pointer to the protocol version number to be operated on
Return Value	None

5.2.14. getBroadcastPingResult

Uint8_t getBroadcastPingResult(int port_num, int protocol_version, int id)	
Functionality	Get the broadcast packet results for the specified port number and protocol version
Parameter	1. port_num: A pointer to the Port Number to be operated on 2. protocol_version: Pointer to the protocol version number to be operated on 3. id: Device ID
Return Value	Returns a pointer to an unsigned 8-bit integer, representing the result of the broadcast packet

5.2.15. reboot

void reboot (int port_num, int protocol_version, uint8_t id)	
Functionality	Restart the device with the specified port number and protocol version
Parameter	1. port_num: Port Number to be operated on 2. protocol_version: The protocol version number to be operated on 3. id: Device ID
Return Value	None

5.2.16. clearMutiTurn

void clearMultiTurn(int port_num, int protocol_version, uint8_t id)	
Functionality	Clear multi-round conversations for the specified port and protocol version
Parameter	1. port_num: Port Number 2. protocol_version: Protocol Version Number 3. id: Device ID
Return Value	None

5.2.17. factoryReset

void factoryReset(int port_num, int protocol_version, uint8_t id, uint8_t option)	
Functionality	Perform a factory reset on devices with the specified port and protocol version
Parameter	1. port_num: Port Number 2. protocol_version: Protocol Version Number 3. id: Device ID 4. option: A pointer to an option, which may be used to specify a specific option or configuration for resetting
Return Value	None

5.2.18. readTx

void readTx(int port_num, int protocol_version, uint8_t id, uint16_t address, uint16_t length)	
Functionality	Read transmission data from a device with a specified port and protocol version
Parameter	1. port_num: Port Number 2. protocol_version: Protocol Version Number 3. id: Device ID 4. address: address of the data to be read 5. length: the length of data to be read
Return Value	None

5.2.19. readRx

void readRx (int port_num, int protocol_version, uint16_t length)	
Functionality	Read received data from a device with a specified port and protocol version
Parameter	1. port_num: A pointer to the Port Number to be operated on 2. protocol_version: Pointer to the protocol version number to be operated on 3. length: A pointer to the length of the data to be read
Return Value	None

5.2.20. readTxRx

void readTxRx (int port_num, int protocol_version, uint8_t id, uint16_t address, uint16_t length)	
Functionality	Simultaneously read transmit and receive data from devices with specified ports and protocol versions
Parameter	1. port_num: Port Number 2. protocol_version: Protocol Version Number 3. id: Device ID 4. address: Address of the data to be read 5. length: length of the data packet to be read
Return Value	None

5.2.21. read1ByteTx

void read1ByteTx (int port_num, int protocol_version, uint8_t id, uint16_t address)	
Functionality	Read one ByteDance of transmitted data from a device with a specified port and protocol version
Parameter	1. port_num: A pointer to the Port Number to be operated on 2. protocol_version: Pointer to the protocol version number to be operated on 3. id: Device ID 4. address: A pointer pointing to the address of the data to be read
Return Value	None

5.2.22. read1ByteRx

uint8_t read1ByteRx(int port_num, int protocol_version)	
Functionality	Read a ByteDance of received data from a device with a specified port and protocol version
Parameter	1. port_num: A pointer to the Port Number to be operated on 2. protocol_version: Pointer to the protocol version number to be operated on
Return Value	Returns the data of a ByteDance read

5.2.23. read1ByteTxRx

uint8_t read1ByteTxRx(int port_num, int protocol_version, uint8_t id, uint16_t address)	
Functionality	Read one ByteDance of sent and received data from a device with a specified port and protocol version
Parameter	1. port_num: A pointer to the Port Number to be operated on 2. protocol_version: Pointer to the protocol version number to be operated on 3. id: Device ID 4. address: A pointer pointing to the address of the data to be read
Return Value	Returns the data of a ByteDance read

5.2.24. read2ByteTx

void read2ByteTx(int port_num, int protocol_version, uint8_t id, uint16_t address)	
Functionality	Read two ByteDance of sent data from the device with the specified port and protocol version
Parameter	1. port_num: Port Number 2. protocol_version: Protocol Version 3. id: Device ID 4. address: A pointer pointing to the address of the data to be read
Return Value	None

5.2.25. read2ByteRx

uint16_t read2ByteRx(int port_num, int protocol_version)	
Functionality	Read two ByteDance of received data from the device with the specified port and protocol version
Parameter	1. port_num: Port Number 2. protocol_version: Protocol Version
Return Value	Return the data of the two bytes read

5.2.26. read2ByteTxRx

uint16_t read2ByteTxRx(int port_num, int protocol_version, uint8_t id, uint16_t address)	
Functionality	Read two ByteDance of sent and received data from the device with the specified port and protocol version
Parameter	1. port_num: A pointer to the Port Number to be operated on 2. protocol_version: Pointer to the protocol version number to be operated on 3. id: Device ID 4. address: A pointer pointing to the address of the data to be read
Return Value	Return the data of the two bytes read

5.2.27. read4ByteTx

void read4ByteTx(int port_num, int protocol_version, uint8_t id, uint16_t address)	
Functionality	Read four ByteDance of transmitted data from the device with the specified port and protocol version
Parameter	1. port_num: Port Number 2. protocol_version: Protocol Version Number 3. id: Device ID 4. address: A pointer pointing to the address of the data to be read
Return Value	None

5.2.28. read4ByteRx

uint32_t read4ByteRx(int port_num, int protocol_version)	
Functionality	Read four ByteDance of received data from a device with a specified port and protocol version
Parameter	1. port_num: Port Number 2. protocol_version: Protocol Version Number
Return Value	Return the data of the four bytes read

5.2.29. read4ByteTxRx

uint32_t read4ByteTxRx(int port_num, int protocol_version, uint8_t id, uint16_t address)	
Functionality	Read four ByteDance of transmitted and received data from the device with the specified port and protocol version
Parameter	1. port_num: Port Number 2. protocol_version: Protocol Version Number 3. id: Device ID 4. address: A pointer pointing to the address of the data to be read
Return Value	Return the data of the four bytes read

5.2.30. writeTxOnly

void writeTxOnly(int port_num, int protocol_version, uint8_t id, uint16_t address, uint16_t length)	
Functionality	Only write and send data to the device with the specified port number and protocol version
Parameter	1. port_num: Port Number 2. protocol_version: Protocol Version Number 3. id: Device ID 4. address: A pointer to the data address to be written 5. length: A pointer to the length of the data to be written
Return Value	None

5.2.31. writeTxRx

void writeTxRx(int port_num, int protocol_version, uint8_t id, uint16_t address, uint16_t length)	
Functionality	Simultaneously write send and receive data to devices with the specified port number and protocol version
Parameter	1. port_num: Pointer to the Port Number to be operated on 2. protocol_version: Pointer to the protocol version number to be operated on 3. id: Device ID 4. address: A pointer to the data address to be written 5. length: A pointer to the length of the data to be written
Return Value	None

5.2.32. write1ByteTxOnly

void write1ByteTxOnly(int port_num, int protocol_version, uint8_t id, uint16_t address, uint8_t data)	
Functionality	Write one byte to the device using the specified port and protocol version, without receiving a response.
Parameters	1. port_num: Pointer to the port number to be used 2. protocol_version: Pointer to the protocol version to be used 3. id: Device ID 4. address: Pointer to the address of the data to be written 5. data: Pointer to the one-byte data to be written
Return Value	None

5.2.33. write1ByteTxRx

void write1ByteTxRx(int port_num, int protocol_version, uint8_t id, uint16_t address, uint8_t data)	
Functionality	Write one byte to the device using the specified port and protocol version, and receive the response.
Parameters	1. port_num:Pointer to the port number to be used 2. protocol_version:Pointer to the protocol version to be used id:Device ID 3. address:Pointer to the address of the data to be written 4. data:Pointer to the one-byte data to be written
Return Value	None

5.2.34. write2ByteTxOnly

void write2ByteTxOnly(int port_num, int protocol_version, uint8_t id, uint16_t address, uint16_t data)	
Functionality	Write only two ByteDance of transmission data to the device with the specified port number and protocol version
Parameter	1. port_num: Pointer to the Port Number to be operated on 2. protocol_version: Pointer to the protocol version number to be operated on 3. id: Device ID 4. address: Pointer to the address of the data to be written 5. data: A pointer to a byte of data to be written
Return Value	None

5.2.35. write2ByteTxRx

void write1ByteTxRx(int port_num, int protocol_version, uint8_t id, uint16_t address, uint8_t data)	
Functionality	Write two ByteDance of transmit and receive data to the device with the specified port number and protocol version simultaneously
Parameter	1. port_num: Pointer to the Port Number to be operated on 2. protocol_version: Pointer to the protocol version number to be operated on 3. id: Device ID 4. address: A pointer to the data address to be written 5. data: Pointer to the two ByteDance data to be written
Return Value	None

5.2.36. write4ByteTxOnly

void write4ByteTxOnly(int port_num, int protocol_version, uint8_t id, uint16_t address, uint32_t data)	
Functionality	Write four bytes to the device using the specified port and protocol version, without receiving a response.
Parameters	1. port_num:Port number 2. protocol_version:Protocol version 3. id:Device ID 4. address:Address of the data to be written 5. data:Pointer to the four-byte data to be written
Return Value	None

5.2.37. write4ByteTxRx

void write4ByteTxRx(int port_num, int protocol_version, uint8_t id, uint16_t address, uint32_t data)	
Functionality	Write four bytes to the device using the specified port and protocol version, and receive the response.
Parameters	1. port_num:Port number 2. protocol_version:Protocol version 3. id:Device ID 4. address:Address of the data to be written 5. data:Pointer to the four-byte data to be written
Return Value	None

5.2.38. regWriteTxOnly

int regWriteTxOnly (PortHandler *port, UINT8_T id, UINT16_T address, UINT16_T length, UINT8_T *data)	
Functionality	Write register data to the device using the specified port and protocol version, without receiving a response.
Parameters	1. port:Target port number 2. id:SIAR ID 3. address:Starting register address 4. length:Packet length 5. data:Pointer to the data array to be sent
Return Value	Return 0 on success, or an error code on failure.

5.2.39. regWriteTxRx

void regWriteTxRx(int port_num, int protocol_version, uint8_t id, uint16_t address, uint16_t length)	
Functionality	Write register data to the device using the specified port and protocol version, and receive the response.
Parameters	1. port:Target port number 2. id:SIAR ID 3. address:Starting register address 4. length:Packet length 5. Data:Pointer to the data array to be sent
Return Value	None

5.2.40. syncReadTx

void syncReadTx(int port_num, int protocol_version, uint16_t start_address, uint16_t data_length, uint16_t param_length)	
Functionality	Send an N-byte synchronous read instruction packet.
Parameters	1. port_num: Port number to be used 2. protocol_version: Protocol version to be used 3. id: Device ID 4. address: Address of the data to be written 5. length: Length of the data to be written
Return Value	None

5.2.41. syncWriteTxOnly

void syncWriteTxOnly(int port_num, int protocol_version, uint16_t start_address, uint16_t data_length, uint16_t param_length)	
Functionality	Synchronously write data to devices using the specified port and protocol version.
Parameters	1. port_num: Port number 2. protocol_version:Protocol version 3. start_address:Pointer to the starting address of the data to be written 4. data_length:Pointer to the length of the data to be written 5. param_length:Pointer to the parameter length
Return Value	None

5.2.42. bulkReadTx

void bulkReadTx(int port_num, int protocol_version, uint16_t param_length)	
Functionality	Perform a bulk read from devices using the specified port and protocol version.
Parameters	1. port_num:Port number to be used 2. protocol_version:Protocol version to be used 3. param_length:Length of the parameters to be read
Return Value	None

5.2.43. bulkWriteTxOnly

void bulkWriteTxOnly(int port_num, int protocol_version, uint16_t param_length)	
Functionality	Perform a bulk write to devices using the specified port and protocol version.
Parameters	1. port_num:Port number 2. protocol_version:Protocol version to be used 3. param_length:Length of the parameters to be written
Return Value	None

5.3. groupSyncWrite.h

5.3.1. GroupSyncWrite

int groupSyncWrite(int port_num, int protocol_version, uint16_t start_address, uint16_t data_length)	
Functionality	Perform a synchronous write operation on the specified port.
Parameters	1. port_num: Port number for the write operation 2. protocol_version:Protocol version to be used 3. start_address: Starting address for the write operation 4. data_length: Length of the data to be written
Return Value	Return 0 on success, or an error code on failure.

5.3.2. groupSyncWriteMakeParam

void groupSyncWriteMakeParam(int group_num)	
Functionality	Create a parameter structure for a synchronous write operation.
Parameters	group_num: Group number for the write operation
Return Value	None

5.3.3. groupSyncWriteAddParam

uint8_t groupSyncWriteAddParam(int group_num, uint8_t id, uint32_t data, uint16_t input_length)	
Functionality	Add parameters to a synchronous write operation.
Parameters	1. group_num:Group number for the write operation 2. id:Data identifier or ID 3. data:Pointer to the data to be written 4. input_length:Input data length
Return Value	Return a pointer to the newly added parameter or data.

5.3.4. groupSyncWriteRemoveParam

void groupBulkWriteRemoveParam(int group_num, uint8_t id)	
Functionality	Remove the specified parameters or data from a synchronous write operation.
Parameters	1. group_num: Group number for the write operation 2. id:ID of the device to be removed
Return Value	None

5.3.5. groupSyncWriteChangeParam

uint8_t groupSyncWriteChangeParam(int group_num, uint8_t id, uint32_t data, uint16_t input_length, uint16_t data_pos)	
Functionality	Change the parameters or data in a synchronous write operation.
Parameters	1. group_num:Group number for the write operation 2. id:ID of the device to be changed 3. data:New data value 4. input_length:Input data length 5. data_pos:Position of the data in the buffer
Return Value	Return a uint8_t value indicating the operation result or status.

5.3.6. groupSyncWriteClearParam

void groupSyncWriteClearParam(int group_num)	
Functionality	Clear all parameters or data in a synchronous write operation.
Parameters	group_num: Integer specifying the group number for the write operation
Return Value	None

5.3.7. groupSyncWriteTxPacket

void groupSyncWriteTxPacket(int group_num)	
Functionality	Send the packet for a synchronous write operation.
Parameters	group_num: Integer specifying the group number for the write operation
Return Value	None

5.4. GroupSyncRead

5.4.1. GroupSyncRead

int groupSyncRead(int port_num, int protocol_version, uint16_t start_address, uint16_t data_length)	
Functionality	Read data using the specified port and protocol version.
Parameters	1. port_num:Port number for the read operation 2. protocol_version:Protocol version to be used 3. start_address:Starting address for the read operation 4. data_length:Pointer to the variable storing the length of the read data
Return Value	Return 0 on success, or an error code on failure.

5.4.2. groupSyncReadMakeParam

void groupSyncReadMakeParam(int group_num)	
Functionality	Set or initialize the parameters for a synchronous read operation.
Parameters	group_num:Group number for the read operation
Return Value	None

5.4.3. groupSyncReadAddParam

uint8_t groupSyncReadAddParam(int group_num, uint8_t id)	
Functionality	Add additional parameters to a synchronous read operation.
Parameters	1. group_num: Group number for the read operation 2. id: ID of the device to be read
Return Value	Return the added parameters or the internal status.

5.4.4. groupSyncReadRemoveParam

void groupSyncReadRemoveParam(int group_num, uint8_t id)	
Functionality	Remove previously added parameters.
Parameters	1. group_num:Group number for the read operation 2. id:ID of the device whose data is to be removed
Return Value	None

5.4.5. groupSyncReadClearParam

void groupSyncReadClearParam(int group_num)	
Functionality	Clear all synchronous read parameters associated with the specified group number.
Parameters	group_num:Group number for the read operation
Return Value	None

5.4.6. groupSyncReadTxPacket

void groupSyncReadTxPacket(int group_num)	
Functionality	Prepare and send the transmission packet for a synchronous read operation.
Parameters	group_num:Group number for the read operation
Return Value	None

5.4.7. groupSyncReadRxPacket

void groupSyncReadRxPacket(int group_num)	
Functionality	Receive and process the response packet for a synchronous read operation.
Parameters	group_num:Group number for the read operation
Return Value	None

5.4.8. groupSyncReadTxRxPacket

void groupSyncReadTxRxPacket(int group_num)	
Functionality	Process the transmission and response packets for a synchronous read operation.
Parameters	group_num:Group number for the read operation
Return Value	None

5.4.9. groupSyncReadIsAvailable

uint8_t groupSyncReadIsAvailable(int group_num, uint8_t id, uint16_t address, uint16_t data_length)	
Functionality	Check whether data is available for the specified group number, ID and address.
Parameters	1. group_num:Group number for the read operation 2. id:ID of the device to be read 3. address: Address to be read 4. data_length:Data length
Return Value	Return the check result.

5.4.10. groupSyncReadGetData

uint32_t groupSyncReadGetData(int group_num, uint8_t id, uint16_t address, uint16_t data_length)	
Functionality	Retrieve the actual data from a synchronous read operation.
Parameters	1. group_num: Integer pointer specifying the group number for the read operation 2. id: Unsigned character pointer to the ID of the device to be read 3. address: Unsigned short integer pointer to the address to be read 4. data_length: Unsigned short integer specifying the data length
Return Value	Return the retrieved data.

5.5. GroupBulkWrite

5.5.1. GroupBulkWrite

int groupBulkWrite(int port_num, int protocol_version)	
Functionality	Perform a bulk write operation by sending multiple packets to the specified port.
Parameters	1. port_num:Port number for the write operation 2. protocol_version:Protocol version
Return Value	Return 0 on success, or a nonzero value on failure.

5.5.2. groupBulkWriteMakeParam

void groupBulkWriteMakeParam(int group_num)	
Functionality	Set or initialize the parameters for a bulk write operation.
Parameters	group_num:Group number for the write operation
Return Value	None

5.5.3. groupBulkWriteAddParam

uint8_t groupBulkWriteAddParam(int group_num, uint8_t id, uint16_t start_address, uint16_t data_length, uint32_t data, uint16_t input_length)	
Functionality	Set or add parameters for a bulk write operation.
Parameters	1. group_num:Group number for the write operation 2. id:ID of the device to be written 3. start_address:Starting address for the write operation 4. data_length:Data length 5. data:Data to be written 6. input_length:Input data length
Return Value	Return 0 on success, or a nonzero value on failure.

5.5.4. groupBulkWriteRemoveParam

void groupBulkWriteRemoveParam(int group_num, uint8_t id)	
Functionality	Remove the specified parameters from a bulk write operation.
Parameters	1. group_num: Group number for the write operation 2. id: ID of the device to be written
Return Value	None

5.5.5. groupBulkWriteChangeParam

uint8_t groupBulkWriteChangeParam(int group_num, uint8_t id, uint16_t start_address, uint16_t data_length, uint32_t data, uint16_t input_length, uint16_t data_pos)	
Functionality	Change the parameters of a bulk write operation.
Parameters	1. group_num: Group number for the write operation 2. id:ID of the device to be written 3. start_address: Starting write address 4. data_length: Data length 5. data:Data to be written 6. input_length:Input data length 7. data_pos: Data position
Return Value	Return 0 on success, or a nonzero value on failure.

5.5.6. groupBulkWriteClearParam

void groupBulkWriteClearParam(int group_num)	
Functionality	Clear the parameters of a bulk write operation.
Parameters	group_num:Group number for the write operation
Return Value	None

5.5.7. groupBulkWriteTxPacket

void groupBulkWriteTxPacket(int group_num)	
Functionality	Send the packet for a bulk write operation.
Parameters	group_num:Group number for the write operation
Return Value	None

5.6. GroupBulkRead

5.6.1. GroupBulkRead

int groupBulkRead(int port_num, int protocol_version)	
Functionality	Perform a bulk read operation.
Parameters	1. port_num:Port number for the read operation 2. protocol_version:Protocol version
Return Value	Return 0 on success, or a nonzero value on failure.

5.6.2. groupBulkReadMakeParam

void groupBulkReadMakeParam(int group_num)	
Functionality	Add parameter storage for a read operation.
Parameters	group_num:Group number for the read operation
Return Value	None

5.6.3. groupBulkReadAddParam

uint8_t groupBulkReadAddParam(int group_num, uint8_t id, uint16_t start_address, uint16_t data_length)	
Functionality	Add parameters to a bulk read operation.
Parameters	1. group_num:Group number for the read operation 2. id:ID of the device to be read 3. start_address:Starting read address 4. data_length:Data length
Return Value	Return 0 on success, or a nonzero value on failure.

5.6.4. groupBulkReadRemoveParam

void groupBulkReadRemoveParam(int group_num, uint8_t id)	
Functionality	Remove the specified parameters from a bulk read operation.
Parameters	1. group_num:Group number for the read operation 2. id:ID of the device to be read
Return Value	None

5.6.5. groupBulkReadClearParam

void groupBulkReadClearParam(int group_num)	
Functionality	Clear the parameters of a bulk read operation.
Parameters	group_num:Group number for the read operation
Return Value	None

5.6.6. groupBulkReadTxPacket

int groupBulkReadTxPacket(int group_num)	
Functionality	Generate the transmission packet for a bulk read operation.
Parameters	group_num:Group number for the read operation
Return Value	Return 0 on success, or a nonzero value on failure.

5.6.7. groupBulkReadRxPacket

int groupBulkReadRxPacket(int group_num)	
Functionality	Receive packets from the specified number of SIAR devices.
Parameters	group_num:Group number for the read operation
Return Value	Return the number of packets received by the specified group.

5.6.8. groupBulkReadTxRxPacket

void groupBulkReadTxRxPacket(int group_num)	
Functionality	Perform a bulk read operation for the specified group number and receive the response.
Parameters	group_num:Group number for the read operation
Return Value	None

5.6.9. groupBulkReadIsAvailable

uint8_t groupBulkReadIsAvailable(int group_num, uint8_t id, uint16_t address, uint16_t data_length)	
Functionality	Check whether a bulk read operation exists in the data storage.
Parameters	1. group_num:Group number for the read operation 2. id:ID of the group for the read operation 3. address:Address of the data to be read 4. data_length:Length of the data to be read
Return Value	None

5.6.10. groupBulkReadGetData

uint32_t groupBulkReadGetData(int group_num, uint8_t id, uint16_t address, uint16_t data_length)	
Functionality	Retrieve data from the received packet.
Parameters	1. group_num:Group number for the read operation 2. id:ID of the group for the read operation 3. address:Address of the data to be read 4. data_length:Length of the data to be read
Return Value	Return a pointer to the data retrieved from the received packet.

1.3 C++ API Interface Description

1. Introduction

This document is a description document for C++ secondary development interfaces

Supported platforms are: Linux, Windows, macOS

2. Function Introduction

The C++ API implements the functions of serial port data reception and transmission, as well as data encapsulation and parsing for SIAR Protocol 1.0 & 2.0, facilitating users to quickly implement communication with SIAR series motors using interface functions. It can be used in Windows, Linux, and macOS environments.

3. Instructions for Use

The contents of the C++ API folder are shown in the figure below, providing three parts: implementation source code, examples, and compiled DLL library files. Among them, the include and src folders contain the implementation source code, the example folder provides examples, and the build folder is the path to the DLL library files generated for each platform.

Name	Modified	Type	Size
build	2024/11/19 14:07	Folder	
example	2024/11/19 14:07	Folder	
include	2024/11/19 14:07	Folder	
src	2024/11/19 14:07	Folder	
keywords.txt	2024/11/15 11:17	Text file	1 KB
library.properties	2024/11/15 11:37	Properties file	1 KB

You can call the C++ API using one of the following two methods:

(1) Copy the implementation source code to the project

(2) Copy the compiled dll library files to the project and add references to the dll libraries

Name	Modified	Type	Size
build	2024/11/19 14:07	Folder	
example	2024/11/19 14:07	Folder	
include	2024/11/19 14:07	Folder	
src	2024/11/19 14:07	Folder	
keywords.txt	2024/11/15 11:17	Text file	1 KB
library.properties	2024/11/15 11:37	Properties file	1 KB

The general steps for using this library are as follows

1. Import Header File

```
#include "siardk.h"                                // Uses siar SDK library
```

2. Define a PortHandler object for data communication and a PacketHandler object responsible for data encapsulation and parsing in protocol format.

```
// Protocol version
#define PROTOCOL_VERSION      2.0          // See which protocol version is used in the siar
#define BAUDRATE                        1000000
#define DEVICENAME                      "/COM5"      // Check which port is being used on your
controller

// Initialize PortHandler instance
// Set the port path
// Get methods and members of PortHandlerLinux or PortHandlerWindows
siar::PortHandler *portHandler =siar::PortHandler::getPortHandler(DEVICENAME);

// Initialize PacketHandler instance
// Set the protocol version
// Get methods and members of Protocol1PacketHandler or Protocol2PacketHandler
siar::PacketHandler *packetHandler = siar::PacketHandler::getPacketHandler(PROTOCOL_VERSION);
```

3. Set the baud rate and open the serial communication device

```
// Open port
  if (portHandler->openPort())
  {
    printf("Succeeded to open the port!\n");
  }
  else
  {
    printf("Failed to open the port!\n");
    printf("Press any key to terminate...\n");
    getch();
    return 0;
  }

  // Set port baudrate
  if (portHandler->setBaudRate(BAUDRATE))
  {
    printf("Succeeded to change the baudrate!\n");
  }
  else
  {
    printf("Failed to change the baudrate!\n");
    printf("Press any key to terminate...\n");
    getch();
    return 0;
  }
```

4. Use the function interface of the packetHandler object to perform data encapsulation, parsing, receiving, and sending functions. For example, the code to ping an online device is as follows.

```
// Try to ping the siar
  // Get siar model number
  siar_comm_result = packetHandler->ping(portHandler, SIAR_ID, &siar_model_number, &siar_error);
  if (siar_comm_result != COMM_SUCCESS)
  {
    printf("%s\n", packetHandler->getTxRxResult(siar_comm_result));
  }
  else if (siar_error != 0)
  {
    printf("%s\n", packetHandler->getRxPacketError(siar_error));
  }
  else
  {
    printf("[ID:%03d] ping Succeeded. siar model number : %d\n", SIAR_ID, siar_model_number);
  }
```

5. Finally, close the serial communication

```
// Close port
  portHandler->closePort();
```

4. Explanation of API Part Macro Definitions

4.1. Communication Status Code Description

Value	Macro Definition	Status Resolution
0	COMM_SUCCESS	Communication success status
-1000	COMM_PORT_BUSY	Port Usage Status
-1001	COMM_TX_FAIL	Status of failed data packet transmission
-1002	COMM_RX_FAIL	Status of failed packet reception
-2000	COMM_TX_ERROR	Transmission Packet Error Status
-3000	COMM_RX_WAITING	Receiving Packet Waiting State
-3001	COMM_RX_TIMEOUT	Receive Packet Timeout Status
-3002	COMM_RX_CORRUPT	Status of corrupted received data packets
-9000	COMM_NOT_AVAILABLE	Status unavailable in Protocol 1.0

4.2. Protocol Instruction Macro Description

Value	Macro Definition	Instruction Parsing
1	INST_PING	ping command value
2	INST_READ	Instruction value of Read
3	INST_WRITE	Written instruction value
4	INST_REG_WRITE	Instruction value written to the register
5	INST_ACTION	Instruction value of the operation
6	INST_FACTORY_RESET	Command value for restoring factory settings
131	INST_SYNC_WRITE	Command value for synchronous write
146	INST_BULK_READ	Instruction values read in batch
8	INST_REBOOT	Instruction value of Reboot
16	INST_CLEAR	Clear command value
85	INST_STATUS	Instruction value of the state
130	INST_SYNC_READ	Instruction value for synchronous read
138	INST_FAST_SYNC_READ	Instruction value for fast synchronous read
147	INST_BULK_WRITE	Instruction value for batch writing

4.3. Device ID Macro Definition

Value	Macro Definition	ID Parsing
0xFE	BROADCAST_ID	Broadcast ID
0xFC	MAX_ID	Maximum ID Value

5. API Interface Function Description

5.1. PortHandler

The PortHandler class is the base class for serial port operations, and its subclasses include PortHandlerArduino, PortHandlerLinux, PortHandlerMac, and PortHandlerWindows, which correspond to different hardware platforms: Arduino, Linux, MacOS, and Windows, respectively, and are responsible for serial communication. The PortHandler class object instances under different hardware platforms can be obtained through the getPortHandler() function interface of the PortHandler class. The main interface functions of the PortHandler class are shown in the following table.

method	Description
getPortHandler	Get the PortHandler instance object
openPort	Open Serial Port
closePort	Close Serial Port
clearPort	Refresh Serial Port
setPortName	Set Device Name
getPortName	Get Device Name
setBaudrate	Set Baud Rate
getBaudrate	Get Baud Rate
getBytesAvailable	Check the number of bytes that can be read in the port buffer
readPort	Read bytes from the port buffer
writePort	Write ByteDance to the port buffer
setPacketTimeout	Set timeout
isPacketTimeout	Check if communication was successful when timeout occurs

5.1.1. getPortHandler

PortHandler * getPortHandler (const char *port_name)	
Functionality	Get the PortHandler instance object
Parameter	port_name: Serial Port Name
Return Value	Pointer to PortHandler instance object

5.1.2. openPort

bool openPort（）	
Functionality	Open Serial Port
Parameter	None
Return Value	true: Successfully opened false: Failed to open

5.1.3. closePort

void closePort()	
Functionality	Close Serial Port
Parameter	None
Return Value	None

5.1.4. clearPort

void clearPort()	
Functionality	Clear Port
Parameter	None
Return Value	None

5.1.5. setPortName

void setPortName(const char* port_name)	
Functionality	Set Device Name
Parameter	port_name: Device Name
Return Value	None

5.1.6. getPortName

char* getPortName()	
Functionality	Get Device Name
Parameter	None
Return Value	Returns the name of the device currently in use.

5.1.7. setBaudrate

bool setBaudrate(const int baudrate)	
Functionality	Set Baud Rate
Parameter	baudrate: target baud rate
Return Value	This function first checks whether the baud rate is available in the selected operating system. If the baud rate is not available, it returns -1.

5.1.8. getBaudrate

int getBaudrate()	
Functionality	Get Baud Rate
Parameter	None
Return Value	Return the previously set baud rate value

5.1.9. getBytesAvailable

int getBytesAvailable()	
Functionality	Check the number of bytes that can be read in the port buffer
Parameter	None
Return Value	Check the amount of data that can be read and return its length

5.1.10. readPort

int readPort(uint8_t *packet, int length)	
Functionality	Read data of the specified number of bytes
Parameter	1. packet: Save the starting address of the read data 1. length: The length of the read bytes
Return Value	Returns the number of bytes actually read

5.1.11. writePort

int writePort(uint8_t *packet, int length)	
Functionality	Write data to the port buffer
Parameter	1. packet: The starting address for storing the data to be written 1. length: Length of data to be written
Return Value	Returns the number of data bytes actually written

5.1.12. setPacketTimeout

void setPacketTimeout(uint16_t packet_length)	
Functionality	Set the timeout based on the number of bytes to be sent, with an internal conversion formula between the number of bytes and the timeout
Parameter	packet_length: ByteDance length of the data to be sent
Return Value	None

5.1.13. setPacketTimeout

void setPacketTimeout(double msec)	
Functionality	Set the communication timeout in milliseconds
Parameter	msec: Timeout
Return Value	None

5.1.14. isPacketTimeout

bool isPacketTimeout()	
Functionality	Check if the communication has timed out
Parameter	None
Return Value	true: Communication did not time out false: Communication timeout

5.2. PacketHandler

The PacketHandler class is the base class for data encapsulation and parsing processing, and its subclasses include the Protocol1PacketHandler class and the Protocol2PacketHandler class, which correspond to the data frame processing of the SIAR Protocol 1.0&2.0 respectively.

By passing the protocol version number (1.0 or 2.0) through the getPacketHandler(float protocol_version) function of the PacketHandler class, an object of the Protocol1PacketHandler class or the Protocol2PacketHandler class is obtained. The interface definitions of the Protocol1PacketHandler class and the Protocol2PacketHandler class are basically the same, and the main interface functions are shown in the following table.

method	Description
getpacketHandler	Get an instance of the PacketHandler object
getProtocolVersion	Get protocol version information
getTxRxResult	Get communication results
getRxPacketError	Error occurred while retrieving and saving hardware information
txPacket	Send data packet
rxPacket	Receive data packets
txRxPacket	Send and receive data packets
ping	Query whether the device is online
broadcastPing	Broadcast query for all online devices
action	Immediately execute the received "register write" command
reboot	Restart SIAR
factoryReset	Restore Factory Settings
readTx	Send a read N ByteDance instruction packet
readRx	Receive and read N ByteDance return data packets
readTxRx	Send and receive N ByteDance data packets
read1ByteTx	Send a read 1 ByteDance instruction packet
read1ByteRx	Receive and read a 1-ByteDance return data packet
read1ByteTxRx	Send and receive 1 ByteDance packet
read2ByteTx	Send a command packet to read 2 ByteDance
read2ByteRx	Receive and read a 2-ByteDance return data packet
read2ByteTxRx	Send and receive 2 ByteDance packets
read4ByteTx	Send a read 4 ByteDance instruction packet

method	Description
read4ByteRx	Receive and read a 4-ByteDance return data packet
read4ByteTxRx	Send and receive 4 ByteDance packets
writeTxOnly	Send a write N ByteDance instruction packet
writeTxRx	Send and receive N ByteDance write packets
write1ByteTxOnly	Send a write 1 ByteDance instruction packet
write1ByteTxRx	Send and receive 1 ByteDance packet
write2ByteTxOnly	Send a write 2 ByteDance instruction packet
write2ByteTxRx	Send and receive 2 ByteDance packets
write4ByteTxOnly	Send a write 4 ByteDance instruction packet
write4ByteTxRx	Send and receive 4 ByteDance packets
regWriteTxOnly	Send a register write instruction data packet
regWriteTxRx	Send and receive register write packets
syncReadTx	Send a synchronous read N ByteDance instruction packet
syncWriteTxOnly	Send a synchronous write N ByteDance instruction packet
bulkReadTx	Send a batch read N ByteDance instruction packet
bulkWriteTxOnly	Send a batch write N ByteDance instruction packet

5.2.1. getpacketHandler

PacketHandler *PacketHandler::getPacketHandler(float protocol_version)	
Functionality	Get an instance of the PacketHandler object
Parameter	protocol_version: Protocol Version (1.0 or 2.0)
Return Value	Pointer to PacketHandler instance object

5.2.2. getProtocolVersion

float getProtocolVersion()	
Functionality	Get protocol version information.
Parameter	None
Return Value	1.0： SIAR Protocol 1.0 2.0： SIAR Protocol 1.0

5.2.3. getTxRxResult

const char *getTxRxResult(int result)	
Functionality	Get the string description of the communication result
Parameter	result: Numerical code of communication result
Return Value	Returns the communication result as a string type

5.2.4. getRxPacketError

const char *getRxPacketError(uint8_t error)	
Functionality	Get the string description of the error in receiving data
Parameter	error: error code
Return Value	Returns the error result as a string type

5.2.5. txPacket

int txPacket (PortHandler *port, uint8_t *txpacket)	
Functionality	Send data packet
Parameter	1. port: PortHandler instance 2. txpacket: Data packet to be sent
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.6. rxPacket

int rxPacket (PorHandler* port,uint8_t* rxpacket)	
Functionality	Receive data packets
Parameter	1. port: PortHandler instance 2. rxpacket: Stores the address of the received packet
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.7. txRxPacket

int txRxPacket(PortHandler *port, uint8_t *txpacket, uint8_t *rxpacket, uint8_t *error = 0)	
Functionality	Send and receive data packets
Parameter	1. port: PortHandler instance 2. txpacket: points to the address of the data to be sent 3. rxpacket: stores received packets 4. error: hardware save error
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.8. ping

int ping（PortHandler *port, uint8_t id, uint8_t *error）	
Functionality	Query whether the device is online
Parameter	1. port: PortHandler instance 2. id: Device ID 3. error: hardware save error
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.9. broadcastPing

int broadcastPing(PortHandler *port, std::vector<uint8_t> &id_list)	
Functionality	Broadcast query for all online devices
Parameter	1. port: PortHandler instance 2. id_list: Stores the list of all online device IDs returned
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.10. action

int action(PortHandler *port, uint8_t id)	
Functionality	Immediately execute the received "register write" command.
Parameter	1. port: PortHandler instance 2. id: Device ID
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.11. reboot

int reboot (PortHandler *port, uint8_t id, uint8_t *error)	
Functionality	Restart the SIAR motor
Parameter	1. port: PortHandler instance 2. id: Device ID 3. error: hardware save error
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.12. factoryReset

int factoryReset (PortHandler *port, uint8_t id, uint8_t option, uint8_t *error)	
Functionality	Restore Factory Settings
Parameter	1. port: PortHandler instance 2. id: Device ID 3. option: Optional parameter used to specify the reset option 0xFF: Reset all parameters 0x01: Reset all parameters except the ID number 0x02: Reset all parameters except ID number and baud rate 4. error: hardware save error
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.13. readTx

int readTx(PortHandler *port, uint8_t id, uint16_t address, uint16_t length)	
Functionality	Send a read N ByteDance instruction packet
Parameter	1. port: PortHandler instance 2. id: Device ID 3. address: register start address 4. length: Packet length
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.14. readRx

int readRx (PortHandler *port, uint16_t length, uint8_t *data, uint8_t *error)	
Functionality	Receive and read N ByteDance return data packets
Parameter	1. port: PortHandler instance 2. length: Packet Length 3. data: Stores the address of the returned data packet 4. error: hardware save error
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.15. readTxRx

int readTxRx (PortHandler *port, uint8_t id, uint16_t address, uint16_t length, uint8_t *data, uint8_t *error)	
Functionality	Send and receive N ByteDance data packets
Parameter	1. port: PortHandler instance 2. id: Device ID 3. address: register start address 4. length: length of the read data 5. data; stores the address of the returned data 6. error: hardware save error
Return Value	Returns the read data upon success, and returns an error code upon failure

5.2.16. read1ByteTx

int read1ByteTx (PortHandler *port, uint8_t id, uint16_t address)	
Functionality	Send a read 1 ByteDance instruction packet
Parameter	1. port: PortHandler instance 2. id: Device ID 3. address: register start address
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.17. read1ByteRx

int read1ByteRx(PortHandler *port, uint8_t *data, uint8_t *error)	
Functionality	Receive and read a 1-ByteDance return data packet
Parameter	1. port: PortHandler instance 2. data: address for storing received data 3. error: hardware save error
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.18. read1ByteTxRx

int read1ByteTxRx (PortHandler *port, uint8_t id,uint16_t address, uint8_t *data, uint8_t *error)	
Functionality	Send and receive 1 ByteDance packet
Parameter	1. port: PortHandler instance 2. id: Device ID 3. address: Register start address 4. data: address for storing received data 5. error: hardware save error
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.19. read2ByteTx

int read2ByteTx (PortHandler *port, uint8_t id, uint16_t address)	
Functionality	Send a command packet to read 2 ByteDance
Parameter	1. port: PortHandler instance 2. id: Device ID 3. address: register start address
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.20. read2ByteRx

int read2ByteRx(PortHandler *port, uint16_t *data, uint8_t *error)	
Functionality	Receive and read a 2-ByteDance return data packet
Parameter	1. port: PortHandler instance 2. data: address for storing received data 3. error: hardware save error
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.21. read2ByteTxRx

int read2ByteTxRx(PortHandler *port, uint8_t id,uint16_t address, UIN16_T *data, uint8_t *error)	
Functionality	Send and receive 2 ByteDance packets
Parameter	1. port: PortHandler instance 2. id: Device ID 3. address: Register start address 4. data: stores the address of received data 5. error: Storage hardware error
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.22. read4ByteTx

int read4ByteTx(PortHandler *port, uint8_t id, uint16_t address)	
Functionality	Send a read 4 ByteDance instruction packet
Parameter	1. port: PortHandler instance 2. id: Device ID 3. address: register start address
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.23. read4ByteRx

int read4ByteRx(PortHandler *port, uint32_t *data, uint8_t *error)	
Functionality	Receive and read a 4-ByteDance return data packet
Parameter	1. port: PortHandler instance 2. data: address for storing received data
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.24. read4ByteTxRx

int read4ByteTxRx (PortHandler *port, UIN8_T id, uint16_t address, uint32_t *data, uint8_t *error)	
Functionality	Send and receive 4 ByteDance packets
Parameter	1. port: PortHandler instance 2. ID: Device ID 3. address: Register start address 4. data: address for storing received data 5. error: hardware save error
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.25. writeTxOnly

int writeTxOnly(PortHandler *port, uint8_t id, uint16_t address, uint16_t length, uint8_t *data)	
Functionality	Send a write N ByteDance instruction packet
Parameter	1. port: PortHandler instance 2. id: Device ID 3. address: register start address 4. length: length of data to be written 5. data: The starting address where the data to be sent is stored
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.26. writeTxRx

int writeTxRx(PortHandler *port, uint8_t id, uint16_t address, uint16_t length, uint8_t *data, uint8_t *error)	
Functionality	Send and receive N ByteDance write packets
Parameter	1. port: PortHandler instance 2. id: Device ID 3. address: register start address 4. length: length of data to be written 5. data: Starting address for storing data to be sent 6. error: save returned hardware error
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.27. write1ByteTxOnly

int write1ByteTxOnly (PortHandler *port, uint8_t id, uint16_t address, uint8_t data)	
Functionality	Send a write 1 ByteDance instruction packet
Parameter	1. port: PortHandler instance 2. id: Device ID 3. address: register start address 4. data: 1 Byte of data to be sent
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.28. write1ByteTxRx

int write1ByteTxRx (PortHandler *port, uint8_t id, uint16_t address, uint8_t data, uint8_t *error)	
Functionality	Send and receive 1 ByteDance packet
Parameter	1. Port: PortHandler instance 2. id: Device ID 3. address: register start address 4. data: 1 Byte of data to be sent 5. error: Save returned hardware error
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.29. write2ByteTxOnly

int write2ByteTxOnly (PortHandler *port, uint8_t id, uint16_t address, uint16_t data)	
Functionality	Send a write 2 ByteDance instruction packet
Parameter	1. port: PortHandler instance 2. id: Device ID 3. address: Register start address 4. data: 2 ByteDance data to be sent
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.30. write2ByteTxRx

int write2ByteTxRx (PortHandler *port, uint8_t id, uint16_t address, uint16_t data, uint8_t *error)	
Functionality	Send and receive 2 ByteDance packets
Parameter	1. port: PortHandler instance 2. id: Device ID 3. address: Register start address 4. data: 2 ByteDance data to be sent 5. error: Save returned hardware error
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.31. write4ByteTxOnly

int write4ByteTxOnly (PortHandler *port, uint8_t id, uint16_t address, uint32_t data)	
Functionality	Send a write 4 ByteDance instruction packet
Parameter	1. port: PortHandler instance 2. data: 4 ByteDance data to be sent
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.32. write4ByteTxRx

int write4ByteTxRx (PortHandler *port, uint8_t id, uint16_t address, uint32_t data, uint8_t *error)	
Functionality	Send and receive 4 ByteDance packets
Parameter	1. port: PortHandler instance 2. id: Device ID 3. address: Register start address 4. data: 4 ByteDance data to be sent 5. error: Save returned hardware error
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.33. regWriteTxOnly

int regWriteTxOnly (PortHandler *port, uint8_t id, uint16_t address, uint16_t length, uint8_t *data)	
Functionality	Send a register write instruction data packet
Parameter	1. port: PortHandler instance 2. id: Device ID 3. address: register start address 4. length: length of data to be written 5. data: Storage address of the data to be sent
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.34. regWriteTxRx

int regWriteTxRx (PortHandler *port, uint8_t id, uint16_t address, uint16_t length, uint8_t *data, uint8_t *error)	
Functionality	Send and receive register write packets
Parameter	1. port: PortHandler instance 2. id: Device ID 3. address: Register start address 4. length: length of data to be written 5. data: Storage address of the data to be sent 6. error: Save returned hardware error
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.35. syncReadTx

int syncReadTx (PortHandler *port, uint16_t address, uint16_t data_length, uint8_t *param, uint16_t param_length)	
Functionality	Send a synchronous read N ByteDance instruction packet
Parameter	1. port: PortHandler instance 2. address: register start address 3. data_length: Data Length 4. param: parameter list 5. param_length: Parameter Length
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.36. syncWriteTxOnly

int syncWriteTxOnly(PortHandler *port, uint16_t start_address, uint16_t data_length, uint8_t *param, uint16_t param_length)	
Functionality	Send a synchronous write N ByteDance instruction packet
Parameter	1. port: PortHandler instance 2. start_address: Register start address 3. data_length: Data Length 4. param: parameter list 5. param_length: Parameter Length
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.37. bulkReadTx

int bulkReadTx (PortHandler *port, uint8_t *param, uint16_t param_length)	
Functionality	Send a batch read N ByteDance instruction packet
Parameter	1. port: PortHandler instance 2. param: parameter list 3. param_length: Parameter Length
Return Value	Returns 0 on success, and returns the communication error code on failure

5.2.38. bulkWriteTxOnly

int bulkWriteTxOnly (PortHandler *port, uint8_t *param, uint16_t param_length)	
Functionality	Send a batch write N ByteDance instruction packet
Parameter	1. port: PortHandler instance 2. param: parameter list 3. param_length: Parameter Length
Return Value	Returns 0 on success, and returns the communication error code on failure

5.3. GroupSyncWrite

The GroupSyncWrite class is responsible for encapsulating the parameters (excluding the start address and data length) of the Sync Write (0x83) command. The main interfaces are shown in the following table.

Method	Description
GroupSyncWrite	constructor function
addParam	Add relevant parameters for the specified device ID
removeParam	Remove the relevant parameters of the specified device ID
changeParam	Modify the relevant parameters of the specified device ID
clearParam	Clear all device parameters
txPacket	Send a Sync Write (0x83) command with encapsulated parameters

The main class member variables are shown in the following table.

Variable Type	Variable Name	Description
PortHandler _	*port	is responsible for serial port data reception and transmission
PacketHandler	*ph_	is responsible for encapsulating and parsing serial port data according to the protocol format
std::vector<uint8_t>	id_list_	Dynamic array, storing the list of devices for synchronous writing
std::map<uint8_t, uint8_t *>	data_list_	An ordered Attribute-Value Pair container that stores data from synchronous write devices. The key is the device ID, and the value is the data storage address of the device to be written to
bool	is_param_changed_	Whether the device data has been modified
uint8_t	*param_	The first address for storing data to be sent

Variable Type	Variable Name	Description
uint16_t	start_address_	Register Start Address
uint16_t	data_length_	Length of data to be written

5.3.1. GroupSyncWrite

GroupSyncWrite(PortHandler *port, PacketHandler *ph, uint16_t start_address, uint16_t data_length)	
Functionality	Constructor function
Parameter	1. port: PortHandler instance 2. ph:PacketHandler instance 3. start_address: Register Start Address 4. data_length: Number of data bytes to be written
Return Value	None

5.3.2. addParam

bool addParam(uint8_t id, uint8_t *data)	
Functionality	Add relevant parameters for the specified device ID
Parameter	1. id: Device ID 2. data: data to be written
Return Value	Returns true if successfully added; returns false if failed

5.3.3. removeParam

void removeParam(uint8_t id)	
Functionality	Remove the relevant parameters of the specified device ID
Parameter	id: Device ID
Return Value	None

5.3.4. changeParam

bool changeParam(uint8_t id, uint8_t *data)	
Functionality	Modify the relevant parameters of the specified device ID
Parameter	1. id: Device ID 2. data: Update and write data
Return Value	false: When the specified ID does not exist in the device ID list, the function will return false true: otherwise return true

5.3.5. clearParam

void clearParam()	
Functionality	Clear all device parameters
Parameter	None
Return Value	None

5.3.6. txPacket

int txPacket(data)	
Functionality	Send a Sync Write (0x83) command with encapsulated parameters
Parameter	None
Return Value	Returns 0 on success, and returns the communication error code on failure

5.4. GroupSyncRead

The GroupSyncRead class is responsible for encapsulating the parameters (excluding the start address and data length) of the Sync Read (0x82) command. The main interfaces are shown in the following table.

Method	Description
GroupSyncRead	Constructor function
addParam	Add the specified device ID to the device list
removeParam	Remove the specified device ID from the device list
clearParam	Clear Device List
txPacket	Send data packet
rxPacket	Receive data packets
txRxPacket	Send and receive data packets
isAvailable	Check if there is any available data among the read data
getData	Retrieve data from the received packet

The main class member variables are shown in the following table.

Variable Type	Variable Name	Description
PortHandler _	*port	is responsible for serial port data reception and transmission
PacketHandler	*ph_	is responsible for encapsulating and parsing serial port data according to the protocol format
std::vector<uint8_t>	id_list_	Dynamic array, storing the list of devices for synchronous read
std::map<uint8_t, uint8_t *>	data_list_	An ordered Attribute-Value Pair container that stores the return data of the synchronous read device. The key is the device ID, and the value is the storage address of the device's return data

Variable Type	Variable Name	Description
std::map<uint8_t, uint8_t *>	error_list_	Ordered Attribute-Value Pair container, storing the returned hardware errors of synchronous read devices. The key is the device ID, and the value is the returned hardware error of the device
bool	last_result_;	Was the last read successful?
bool	is_param_changed_	Whether the device data has been modified
uint8_t	*param_	The first address for storing data to be sent
uint16_t	start_address_	Register Start Address
uint16_t	data_length_	Length of data to be read

5.4.1. GroupSyncRead

GroupSyncRead(PortHandler *port, PacketHandler *ph, uint16_t start_address, uint16_t data_length)	
Functionality	Constructor function
Parameter	1. port: PortHandler instance 2. ph: Protocol1PacketHandler instance or an instance of Protocol2PacketHandler 1. start_address: Register start address 2. data_length: Data Length
Return Value	None

5.4.2. addParam

bool addParam(uint8_t id)	
Functionality	Add the specified device ID to the device list
Parameter	id: Device ID
Return Value	false: When using Protocol 1.0 or the specified device ID is already in the device list true: Successfully added

5.4.3. removeParam

void removeParam(uint8_t id)	
Functionality	Remove the specified device ID from the device list
Parameter	id: Device ID
Return Value	false: When using Protocol 1.0 or when the specified device ID does not exist in the device list true: Successfully removed

5.4.4. clearParam

void clearParam()	
Functionality	Clear Device List
Parameter	None
Return Value	None

5.4.5. txPacket

int txPacket()	
Functionality	Send data packet
Parameter	None
Return Value	Returns 0 on success, and returns the communication error code on failure

5.4.6. rxPacket

int rxPacket()	
Functionality	Receive data packets
Parameter	None
Return Value	Returns 0 on success, and returns the communication error code on failure When successfully read, the data returned by the device is stored in the member variable data_list_, and the returned hardware error is stored in the member variable error_list_

5.4.7. txRxPacket

int txRxPacket()	
Functionality	Send and receive data packets
Parameter	None
Return Value	Returns 0 on success, and returns the communication error code on failure When successfully read, the data returned by the device is stored in the member variable data_list_, and the returned hardware error is stored in the member variable error_list_

5.4.8. isAvailable

isAvailable(id, address, data_length)	
Functionality	Check if there is any available data among the read data
Parameter	1. id: Device ID 1. address: register start address 2. data_length: Data Length
Return Value	false: No data available true: There is data of length data_length starting from the specified memory table address address

5.4.9. getData

getData(id, address, data_length)	
Functionality	Retrieve data from the received packet
Parameter	1. id: Device ID 2. address: Register start address 3. data_length: Data Length
Return Value	0: No data was retrieved Other: Read data value

5.5. GroupBulkWrite

The GroupBulkWrite class is responsible for encapsulating the parameters of the Bulk Write (0x93) command. The main interfaces are shown in the following table.

Method	Description
GroupSyncWrite	Constructor function
addParam	Add relevant parameters for the specified device ID
removeParam	Remove the relevant parameters of the specified device ID
changeParam	Modify the relevant parameters of the specified device ID
clearParam	Clear all device parameters
txPacket	Send a synchronous write Bulk Write (0x93) command with encapsulated parameters

The main class member variables are shown in the following table.

Variable Type	Variable Name	Description
PortHandler _	*port	is responsible for serial port data reception and transmission
PacketHandler	*ph_	is responsible for encapsulating and parsing serial port data according to the protocol format
std::vector<uint8_t>	id_list_	Dynamic array, storing the list of devices for synchronous writing
std::map<uint8_t, uint16_t>	address_list_	<id, start_address> An ordered Attribute-Value Pair container that stores the starting register addresses of devices for batch writing. The key is the device ID, and the value is the starting register address of the device to be written
std::map<uint8_t,uint16_t>	length_list_	<id, data_length> An ordered Attribute-Value Pair container that stores the length of data to be written for batch write devices. The key is the device ID, and the value is the length of data to be written to the device
Variable Type	Variable Name	Description
std::map<uint8_t, uint8_t *>	data_list_	<id, data> An ordered Attribute-Value Pair container that stores data for batch writing to devices.

Variable Type	Variable Name	Description
		The key is the device ID, and the value is the data storage address of the device to be written to
bool	is_param_changed_	Whether the device data has been modified
uint8_t	*param_	The first address for storing data to be sent
uint16_t	param_length_	Length of data to be sent

5.5.1. groupBulkWrite

GroupBulkWrite(PortHandler *port, PacketHandler *ph)	
Functionality	Constructor function
Parameter	1. port: PortHandler instance 2. ph:Protocol1PacketHandler instance or an instance of Protocol2PacketHandler
Return Value	None

5.5.2. addParam

bool addParam(uint8_t id, uint16_t start_address, uint16_t data_length, uint8_t *data)	
Functionality	Add relevant parameters for the specified device ID
Parameter	1. id: Device ID 2. start_address: Register start address 3. data_length: length of data to be written 4. data: data to be written
Return Value	false: Addition failed ture: Added successfully

5.5.3. removeParam

void removeParam(uint8_t id)	
Functionality	Remove the relevant parameters of the specified device ID
Parameter	id: Device ID
Return Value	false: When no device data is specified in the parameter device list, the function returns false, true: Successfully removed

5.5.4. changeParam

bool changeParam(uint8_t id, uint16_t start_address, uint16_t data_length, uint8_t *data)	
Functionality	Modify the relevant parameters of the specified device ID
Parameter	1. id: Device ID 2. start_address: The starting address of the register where data is to be written 3. data_length: length of data to be written 4. data: data to be written
Return Value	false: When the target ID does not exist in the ID list, the function will return false true: Successfully modified

5.5.5. clearParam

void clearParam()	
Functionality	Clear all device parameters
Parameter	None
Return Value	None

5.5.6. txPacket

int txPacket()	
Functionality	Send a synchronous write Bulk Write (0x93) command with encapsulated parameters
Parameter	None
Return Value	Returns 0 on success, and returns the communication error code on failure

5.6. GroupBulkRead

The GroupBulkRead class is responsible for encapsulating the parameters of the Bulk Write (0x92) command for batch reading. The main interfaces are shown in the following table.

Method	Description
GroupBulkRead	Constructor function
addParam	Add relevant parameters for the target device ID
removeParam	Remove relevant parameters of the target device ID
clearParam	Clear all device parameters
txPacket	Send data packet
rxPacket	Receive data packets
txRxPacket	Send and receive data packets
isAvailable	Check if there is any available data among the read data
getData	Retrieve data from the received packet

The main class member variables are shown in the following table.

Variable Type	Variable Name	Description
PortHandler _	*port	is responsible for serial port data reception and transmission
PacketHandler	*ph_	is responsible for encapsulating and parsing serial port data according to the protocol format
std::vector<uint8_t>	id_list_	Dynamic array, storing the list of devices for synchronous writing
std::map<uint8_t, uint16_t>	address_list_	<id, start_address> An ordered Attribute-Value Pair container that stores the starting register addresses of devices for batch reading. The key is the device ID, and the value is the starting register address of the device to be read
std::map<uint8_t,uint16_t>	length_list_	<id, data_length> An ordered Attribute-Value Pair container that stores the data length to be read for batch read devices. The key is the device ID, and the value is the data length of the device to be read
std::map<uint8_t, uint8_t *>	data_list_	<id, data> An ordered Attribute-Value Pair container that stores the return data of batch read devices. The key is the device ID, and the value is the storage address of the return data of the device to be read
std::map<uint8_t, uint8_t *>	error_list_	<id, error> An ordered Attribute-Value Pair container that stores the returned hardware errors of batch read devices. The key is the device ID, and the value is the returned hardware error
bool	last_result_	Was the last read successful?
bool	is_param_changed_	Whether the device data has been modified
uint8_t	*param_	The first address for storing data to be sent

5.6.1. GroupBulkRead

GroupBulkRead(PortHandler *port, PacketHandler *ph)	
Functionality	Constructor function
Parameter	1. port: PortHandler instance 2. ph:Protocol1PacketHandler instance or an instance of Protocol2PacketHandler
Return Value	None

5.6.2. addParam

bool addParam(uint8_t id, uint16_t start_address, uint16_t data_length)	
Functionality	Add relevant parameters for the target device ID
Parameter	1. id: Device ID 2. start_address: The starting address of the register for the data to be read 3. data_length: Data Length
Return Value	true: Added successfully false: Addition failed

5.6.3. removeParam

void removeParam(uint8_t id)	
Functionality	Remove relevant parameters of the target device ID
Parameter	id: Device ID
Return Value	None

5.6.4. clearParam

void clearParam()	
Functionality	Clear all device parameters
Parameter	id: Device ID
Return Value	None

5.6.5. txPacket

int txPacket()	
Functionality	Send data packet
Parameter	None
Return Value	Returns 0 on success, and returns the communication error code on failure

5.6.6. rxPacket

int rxPacket()	
Functionality	Receive data packets
Parameter	None
Return Value	Returns 0 on success, and returns the communication error code on failure When successfully read, the data returned by the device is stored in the member variable data_list_, and the returned hardware error is stored in the member variable error_list_

5.6.7. txRxPacket

int txRxPacket()	
Functionality	Send and receive data packets
Parameter	None
Return Value	Returns 0 on success, and returns the communication error code on failure When successfully read, the data returned by the device is stored in the member variable data_list_, and the returned hardware error is stored in the member variable error_list_

5.6.8. isAvailable

bool isAvailable(uint8_t id, uint16_t address, uint16_t data_length)	
Functionality	Check if there is any available data among the read data
Parameter	1. id: Device ID 2. address: Register start address 3. data_length: Data Length
Return Value	false: Returns false when there is no data from the target address true: otherwise return true

5.6.9. getData

uint32_t getData(uint8_t id, uint16_t address, uint16_t data_length)	
Functionality	Retrieve data from the received packet
Parameter	1. id: Device ID 2. address: Register start address 3. data_length: Data Length
Return Value	0: No data was obtained Other: Read data value

1.4 ROS & Python API Interface Description

The ROS package organizes Python API source files in the form of a package, so using the SDK API in the ROS environment is consistent with using the Python API interface independently.

1. Introduction

This document is a description document for Python secondary development interfaces

Supported platforms are: Linux, Windows

Python Version: Python 3 or higher

2. Function Introduction

The Python API implements functions for serial data reception and transmission, as well as data encapsulation and parsing for SIAR Protocol 1.0 & 2.0, facilitating users to quickly implement communication with SIAR series motors using interface functions. It can be used in both Windows and Linux environments.

3. Instructions for Use

The Python API implementation code is stored in the siardk folder, and the content is as follows:

Name	Modified	Type	Size
__init__.py	2024/11/15 13:59	Python file	1 KB
group_bulk_read.py	2024/11/15 13:59	Python file	6 KB
group_bulk_write.py	2024/11/15 13:59	Python file	4 KB
group_sync_read.py	2024/11/15 13:59	Python file	5 KB
group_sync_write.py	2024/11/15 13:59	Python file	3 KB
packet_handler.py	2024/11/15 13:59	Python file	2 KB
port_handler.py	2024/11/15 13:59	Python file	4 KB
protocol1_packet_handler.py	2024/11/15 13:59	Python file	18 KB
protocol2_packet_handler.py	2024/11/15 13:59	Python file	31 KB
vkrobot_def.py	2024/11/15 13:59	Python file	2 KB

1. group_bulk_read.py: Encapsulation and parsing of the bulk_read (0x92) instruction for batch reading

2. group_bulk_write.py: Encapsulation and parsing of batch write bulk_write (0x93) instructions

3. group_sync_read.py: Encapsulation and parsing of the sync_read (0x82) instruction for synchronous reading

4. group_sync_write.py: Encapsulation and parsing of the sync_write (0x83) instruction for synchronous writing

5. packe_handle.py: Returns an instance object of the SIAR Protocol 1.0 or 2.0 protocol encapsulation class based on the input protocol version parameter

6. port_handler.py: To operate hardware devices (open and close), and implement data sending and receiving

7. protocol1_packet_handler.py: Data encapsulation and parsing for SIAR Protocol1.0 protocol frame format

8. protocol2_packet_handler.py: Encapsulation and Parsing of SIAR Protocol2.0 Protocol Frame Format Data

The general steps for using this Python API module are as follows:

1. Importing Modules: Use the import statement to import modules so that you can use them in your code. For example:

```
from siardk import *
```

2. Define a PortHandler object for data communication, and call the PacketHandler function to obtain the data encapsulation and parsing class object corresponding to the protocol version.

```
DEVICENAME                  = '/dev/ttyUSB0'
# Protocol version
PROTOCOL_VERSION            = 2.0
# Initialize PortHandler instance
# Set the port path
# Get methods and members of PortHandlerLinux or PortHandlerWindows
portHandler = PortHandler(DEVICENAME)

# Initialize PacketHandler instance
# Set the protocol version
# Get methods and members of Protocol1PacketHandler or Protocol2PacketHandler
packetHandler = PacketHandler(PROTOCOL_VERSION)
# Set the baud rate and open the serial communication device
if portHandler.openPort():
    print("Succeeded to open the port")
else:
print("Failed to open the port")
if portHandler.setBaudRate(BAUDRATE):
    print("Succeeded to change the baudrate")
else:
print("Failed to change the baudrate")
```

3. Set the baud rate and open the serial communication device

```
if portHandler.openPort():
    print("Succeeded to open the port")
else:
print("Failed to open the port")
if portHandler.setBaudRate(BAUDRATE):
    print("Succeeded to change the baudrate")
else:
print("Failed to change the baudrate")
```

4. Use the function interface of the packetHandler object to perform data encapsulation, parsing, reception, and transmission functions.

```
# Try to ping the siar
# Get siar model number
siar_model_number, siar_comm_result, siar_error = packetHandler.ping(portHandler, SIAR_ID)
if siar_comm_result != COMM_SUCCESS:
    print("%s" % packetHandler.getTxRxResult(siar_comm_result))
elif siar_error != 0:
    print("%s" % packetHandler.getRxPacketError(siar_error))
else:
    print("[ID:%03d] ping Succeeded. siar model number : %d" % (SIAR_ID, siar_model_number))
```

5. Finally, close the serial communication

```
// Close port
  portHandler->closePort();
```

4. Explanation of API Part Macro Definitions

4.1. Communication Status Code Description

Value	Macro Definition	Status Resolution
0	COMM_SUCCESS	Communication success status
-1000	COMM_PORT_BUSY	Port Usage Status
-1001	COMM_TX_FAIL	Status of failed data packet transmission
-1002	COMM_RX_FAIL	Status of failed packet reception
-2000	COMM_TX_ERROR	Transmission Packet Error Status
-3000	COMM_RX_WAITING	Receiving Packet Waiting State
-3001	COMM_RX_TIMEOUT	Receive Packet Timeout Status
-3002	COMM_RX_CORRUPT	Status of corrupted received data packets
-9000	COMM_NOT_AVAILABLE	Status unavailable in Protocol 1.0

4.2. Protocol Instruction Macro Description

Value	Macro Definition	Instruction Parsing
1	INST_PING	ping command value
2	INST_READ	Instruction value of Read
3	INST_WRITE	Written instruction value
4	INST_REG_WRITE	Instruction value written to the register
5	INST_ACTION	Instruction value of the operation
6	INST_FACTORY_RESET	Command value for restoring factory settings
131	INST_SYNC_WRITE	Command value for synchronous write
146	INST_BULK_READ	Instruction values read in batch
8	INST_REBOOT	Instruction value of Reboot
16	INST_CLEAR	Clear command value
85	INST_STATUS	Instruction value of the state
130	INST_SYNC_READ	Instruction value for synchronous read
138	INST_FAST_SYNC_READ	Instruction value for fast synchronous read
147	INST_BULK_WRITE	Instruction value for batch writing

4.3. Device ID Macro Definition

Value	Macro Definition	ID Parsing
0xFE	BROADCAST_ID	Broadcast ID
0xFC	MAX_ID	Maximum ID Value

5. API Interface Function Description

5.1. PortHandler

The PortHandler class is a serial port operation class, and its main interface functions are shown in the following table.

method	Description
PortHandler()	Initialize the serial port object
openPort()	Open Serial Port
closePort()	Close Serial Port
clearPort()	Refresh Serial Port
setPortName()	Set Device Name
getPortName()	Get Device Name
setBaudrate()	Set Baud Rate
getBaudrate()	Get Baud Rate
getBytesAvailable()	Check the number of bytes that can be read in the port buffer
readPort()	Read bytes from the port buffer
writePort()	Write ByteDance to the port buffer
setPacketTimeout()	Set timeout
setPacketTimeoutMillis()	Set timeout (in milliseconds)
isPacketTimeout()	Check if communication is successful when timeout occurs

5.1.1. PortHandler

PortHandler(port_name)	
Functionality	Constructor function
Parameter	port_name: Serial Port Name
Return Value	None

5.1.2. openPort

openPort	
Functionality	Open Serial Port
Parameter	None
Return Value	True: Successfully opened False: Failed to open

5.1.3. closePort

closePort()	
Functionality	Close Serial Port
Parameter	None
Return Value	None

5.1.4. clearPort

clearPort()	
Functionality	Clear Port
Parameter	None
Return Value	None

5.1.5. setPortName

setPortName(port_name)	
Functionality	Set Device Name
Parameter	port_name: Used to pass in a serial port name
Return Value	None

5.1.6. getPortName

getPortName()	
Functionality	Get Device Name
Parameter	None
Return Value	This function returns the name of the device that the port is currently using.

5.1.7. setBaudrate

setBaudrate(baudrate)	
Functionality	Set Baud Rate
Parameter	baudrate: target baud rate
Return Value	This function first checks whether the baud rate is available in the selected operating system. If the baud rate is not available, it returns -1.

5.1.8. getBaudrate

getBaudrate()	
Functionality	Get Baud Rate
Parameter	None
Return Value	Return the previously set baud rate value

5.1.9. getBytesAvailable

getBytesAvailable()	
Functionality	Check the number of bytes that can be read in the port buffer
Parameter	None
Return Value	Check the amount of data that can be read and return its length

5.1.10. readPort

readPort(length)	
Functionality	Read data of the specified number of bytes
Parameter	length: The length of the read bytes
Return Value	Returns the number of bytes actually read

5.1.11. writePort

writePort(packet)	
Functionality	Write data to the port buffer
Parameter	packet: Data packet to be written
Return Value	Transmit ByteDance data and return the amount of data written

5.1.12. setPacketTimeout

setPacketTimeout(packet_length)	
Functionality	Set the timeout based on the number of bytes to be sent, with an internal conversion formula between the number of bytes and the timeout
Parameter	packet_length: ByteDance length of the data to be sent
Return Value	None

5.1.13. setPacketTimeoutMillis

setPacketTimeoutMillis(msec)	
Functionality	Set timeout (in milliseconds)
Parameter	msec: Set the timeout for packet transmission to prepare for determining the communication result
Return Value	None

5.1.14. isPacketTimeout

isPacketTimeout()	
Functionality	Check if the communication has timed out
Parameter	None
Return Value	True: Communication did not time out False: Communication timeout

5.2. PacketHandler

The function PacketHandler(protocol_version) obtains an object of the Protocol1PacketHandler class or an object of the Protocol2PacketHandler class based on the passed-in protocol version number (1.0 or 2.0), corresponding to the data frame processing of SIAR Protocol 1.0 and 2.0 respectively. The interface definitions of the Protocol1PacketHandler class and the Protocol2PacketHandler class are basically the same, and the main interface functions are shown in the following table.

method	Description
getTxRxResult	Get communication results
getRxPacketError	Get hardware error
txPacket	Send data packet
rxPacket	Receive data packets
txRxPacket	Send and receive data packets
ping	Query whether the device is online
broadcastPing	Broadcast query for all online devices
action	Immediately execute the received "register write" command
regWrite	Write the data packet and wait for the 'Action' command
reboot	Restart SIAR
factoryReset	Reset all SIAR settings
readTx	Send an N-byte read command data packet
readRx	Receive N ByteDance read status data packets
readTxRx	Send and receive N ByteDance packets
read1ByteTx	Send a 1 ByteDance read instruction packet
read1ByteRx	Receive a 1 ByteDance read status data packet
read1ByteTxRx	Send and receive 1 ByteDance packet
read2ByteTx	Send a 2 ByteDance read instruction packet
read2ByteRx	Receive a 2 ByteDance read status data packet
read2ByteTxRx	Send and receive 2 ByteDance packets
read4ByteTx	Send a 4-ByteDance read instruction packet
read4ByteRx	Receive a 4 ByteDance read status data packet
read4ByteTxRx	Send and receive 4 ByteDance packets
writeTxOnly	Send N ByteDance write command data packet
writeTxRx	Send and receive N ByteDance packets
write1ByteTxOnly	Send a 1 ByteDance write command data packet
write1ByteTxRx	Send and receive 1 ByteDance packet
write2ByteTxOnly	Send a 2 ByteDance write command data packet
write2ByteTxRx	Send and receive 2 ByteDance packets

method	Description
write4ByteTxOnly	Send a 4-ByteDance write command data packet
write4ByteTxRx	Send and receive 4 ByteDance packets
regWriteTxOnly	Send register write instruction data packet
regWriteTxRx	Send and receive register write packets
syncReadTx	Send N ByteDance synchronous read command data packet
syncWriteTxOnly	Send N ByteDance synchronous write command data packet
bulkReadTx	Send N ByteDance batch read instruction data packets
bulkWriteTxOnly	Send N ByteDance batch write command data packets

5.2.1. getTxRxResult

getTxRxResult(result)	
Functionality	Get the string description of the communication result
Parameter	result: Numerical code of communication result
Return Value	Returns the communication result as a string type

5.2.2. getRxPacketError

getRxPacketError(error)	
Functionality	Get the string description of the error in receiving data
Parameter	error: error code
Return Value	Returns the error result as a string type

5.2.3. txPacket

txPacket (port, txpacket)	
Functionality	Send data packet
Parameter	①port PortHandler Instance ②txpacket Data packets to be sent
Return Value	(result) result: communication result

5.2.4. rxPacket

rxPacket (port)	
Functionality	Receive data packets
Parameter	①port PortHandler Instance
Return Value	(rxpacket,result) rxpacket: Received Packet result: communication result

5.2.5. txRxPacket

txRxPacket(port, txpacket)	
Functionality	Send and receive data packets
Parameter	①port PortHandler instance ②Txpacket Transmit Data Packet
Return Value	(rxpacket,result,error) rxpacket: received data packet result: communication result error: hardware error

5.2.6. ping

ping(port, broadcast_id)	
Functionality	Query whether the device is online
Parameter	①port PortHandler instance ②siar_id Device ID
Return Value	(result, error, model_number) result: communication result error: hardware error model_number: Motor Number

5.2.7. broadcastPing

broadcastPing(port)	
Functionality	Broadcast query for all online devices
Parameter	①port PortHandler instance
Return Value	(result, data_list) result: communication result data_list: ID list of online devices

5.2.8. action

action(port, siar_id)	
Functionality	Immediately execute the received "register write" command.
Parameter	①port PortHandler instance ②siar_id Device ID
Return Value	(result) result: communication result

5.2.9. reboot

reboot (port, siar_id)	
Functionality	Restart the SIAR motor
Parameter	①port PortHandler Instance ②siar_id Device ID
Return Value	(result, error) result: communication result error: hardware error

5.2.10. factoryReset

factoryReset (port, siar_id, option)	
Functionality	Restore Factory Settings
Parameter	①port PortHandler Instance ②siar_id Device ID ③ option 0xFF: Reset all parameters 0x01: Reset all parameters except the ID number 0x02: Reset all parameters except ID number and baud rate
Return Value	(result, error) result: communication result error: hardware error

5.2.11. readTx

readTx(port, siar_id, address,length)	
Functionality	Send a command to read register data
Parameter	①port PortHandler Instance ②siar_id Device ID ③address Start address of the register to be read ④length Length of the data packet to be read
Return Value	(result) result: communication result

5.2.12. readRx

readRx (port, siar_id,length)	
Functionality	Receive the return data of the instruction to read register data
Parameter	①port PortHandler Instance ②siar_id Device ID ③length Read the length of the data packet
Return Value	(data, result, error) data: received data packet result: communication result error: hardware error

5.2.13. readTxRx

readTxRx (port, siar_id, address,length)	
Functionality	Send a read register data instruction and receive the returned data
Parameter	①port PortHandler instance ②siar_id Device ID ③address Start address of the register to be read ④length Length of the data packet to be read
Return Value	(data, result, error) data: Return packet received result: communication result error: hardware error

5.2.14. read1ByteTx

read1ByteTx (port, siar_id, address)	
Functionality	Send a command to read 1 ByteDance register data
Parameter	①port PortHandler instance ②siar_id Device ID ③address Start address of the register to be read
Return Value	(result) result: communication result

5.2.15. read1ByteRx

read1ByteRx(port, siar_id)	
Functionality	Receive the return data of the instruction to read 1 ByteDance register data
Parameter	①port PortHandler instance ②siar_id Device ID
Return Value	(data_read, result, error) data_read: Returned data received result: communication result error: hardware error

5.2.16. read1ByteTxRx

read1ByteTxRx (port, siar_id,address)	
Functionality	Send a command to read 1 ByteDance register data and receive the returned data
Parameter	①port PortHandler instance ②siar_id Device ID ③address Start address of the register to be read
Return Value	(data_read, result, error) data_read: Returned data received result: communication result error: hardware error

5.2.17. read2ByteTx

read2ByteTx (port, siar_id, address)	
Functionality	Send a command to read 2 ByteDance register data
Parameter	①port PortHandler instance ②siar_id Device ID ③address Start address of the register to be read
Return Value	(result) result: communication result

5.2.18. read2ByteRx

Read2ByteRx(port, siar_id)	
Functionality	Receive and read the return data of the instruction to read 2 ByteDance register data
Parameter	①port PortHandler instance ②siar_id Device ID
Return Value	(data_read, result, error) data_read: received data result: communication result error: hardware error

5.2.19. read2ByteTxRx

Read2ByteTxRx(port, siar_id, address)	
Functionality	Send a command to read 2 ByteDance register data and receive the returned data
Parameter	①port PortHandler instance ②siar_id Device ID ③address Start address of the register to be read
Return Value	(data_read, result, error) data_read: Received data result: communication result error: hardware error

5.2.20. read4ByteTx

Read4ByteTx(port, siar_id, address)	
Functionality	Send a command to read 4 ByteDance register data
Parameter	①port PortHandler instance ②siar_id Device ID ③address Start address of the register to be read
Return Value	(result) result: communication result

5.2.21. read4ByteRx

read4ByteRx(port, siar_id)	
Functionality	Receive the return data of the instruction to read 4 ByteDance register data
Parameter	①port PortHandler instance ②siar_id Device ID
Return Value	(data_read, result, error) data_read: Received data result: communication result error: hardware error

5.2.22. read4ByteTxRx

read4ByteTxRx (port, siar_id,address)	
Functionality	Send a command to read 4 ByteDance register data and receive the returned data
Parameter	①port PortHandler instance ②siar_id Device ID ③address Start address of the register to be read
Return Value	(data_read, result, error) data_read: Received data result: communication result error: hardware error

5.2.23. writeTxOnly

writeTxOnly(port, siar_id, address,length,data)	
Functionality	Send write register data instruction
Parameter	①port PortHandler instance ②siar_id Device ID ③address Start address of the register to be written with data ④length Length of the data packet to be written ⑤data Data to be written to the register
Return Value	(result) result: communication result

5.2.24. writeTxRx

writeTxRx(port, siar_id, address,length,data)	
Functionality	Send the write register data instruction and receive the return
Parameter	①port PortHandler instance ②siar_id Device ID ③address Start address of the register to be written ④length Length of the data packet to be written ⑤data Data to be written to the register
Return Value	(result, error) result: communication result error: hardware error

5.2.25. write1ByteTxOnly

write1ByteTxOnly (port, siar_id, address,data)	
Functionality	Send a command to write 1 ByteDance register data
Parameter	①port PortHandler instance ②siar_id Device ID ③address Start address of the register to be written with data ④data Data for transmission
Return Value	(result) result: communication result

5.2.26. write1ByteTxRx

write1ByteTxRx (port, siar_id, address,data)	
Functionality	Send a write 1 ByteDance register data instruction and receive the return
Parameter	①port PortHandler instance ②siar_id Device ID ③address Start address of the register to be written with data ④data Data for transmission
Return Value	(result, error) result: communication result error: hardware error

5.2.27. write2ByteTxOnly

write2ByteTxOnly (port, siar_id, address,data)	
Functionality	Send a write 2 ByteDance register data instruction
Parameter	①port PortHandler instance ②siar_id Device ID ③address Start address of the register to be written with data ④data Data for transmission
Return Value	(result) result: communication result

5.2.28. write2ByteTxRx

write2ByteTxRx (port, siar_id, address,data)	
Functionality	Send a write 2 ByteDance register data instruction and receive the return
Parameter	①port PortHandler instance ②siar_id Device ID ③address Start address of the register to be written with data ④data Data for transmission
Return Value	(result, error) result: communication result error: hardware error

5.2.29. write4ByteTxOnly

write4ByteTxOnly (port, siar_id, address,data)	
Functionality	Send a write 4 ByteDance register data instruction
Parameter	①port PortHandler instance ②siar_id Device ID ③address Start address of the register to be written with data ④data Data for transmission
Return Value	(result) result: communication result

5.2.30. write4ByteTxRx

write4ByteTxRx (port, siar_id, address,data)	
Functionality	Send a write 4 registers data instruction and receive the return
Parameter	①port PortHandler instance ②siar_id Device ID ③address Start address of the register to be written with data ④data Data for transmission
Return Value	(result, error) result: communication result error: hardware error

5.2.31. regWriteTxOnly

regWriteTxOnly (port, siar_id, address,length,data)	
Functionality	Send register write instruction to write register data
Parameter	①port PortHandler instance ②siar_id Device ID ③address Start address of the register to be written with data ④length Length of Packet to be Sent ⑤data Data to be sent for transmission
Return Value	(result) result: communication result

5.2.32. regWriteTxRx

regWriteTxRx (port, siar_id, address,length,data)	
Functionality	Send the register write instruction to write register data and receive the return
Parameter	①port PortHandler instance ②siar_id Device ID ③address Start address of the register to be written with data ④length Packet Length ⑤data Data for transmission
Return Value	(result, error) result: communication result error: hardware error

5.2.33. syncReadTx

syncReadTx (port, start_address, data_length, param, param_length)	
Functionality	Send a synchronous read register data instruction
Parameter	①port PortHandler instance ②start_address Start address of the register for data to be read ③data_length Length of data to be read ④param Parameters of the device ID list to be read ⑤param_length Length of the list of device IDs to be read
Return Value	(result) result: communication result

5.2.34. syncWriteTxOnly

syncReadTxOnly (port, start_address, data_length, param, param_length)	
Functionality	Send synchronous write register data instruction
Parameter	①port PortHandler instance ②start_address Start address of the register to be written with data ③data_length Data Length ④param Data to be written ⑤param_length Length of data to be written
Return Value	(result) result: communication result

5.2.35. bulkReadTx

bulkReadTx (port, param, param_length)	
Functionality	Send batch read register data instruction
Parameter	①port PortHandler instance ②param Device (Device ID, Register Start Address, Read Length, etc.) Parameters ③param_length param Parameter Length
Return Value	(result) result: communication result

5.2.36. bulkWriteTxOnly

bulkReadTxOnly (port, param, param_length)	
Functionality	Send batch write register data instruction
Parameter	①port PortHandler instance ②param Device (device ID, register start address, data to be written, etc.) parameters ③param_length param Parameter Length
Return Value	(result) result: communication result

5.3. GroupSyncWrite

The GroupSyncWrite class is responsible for encapsulating the parameters (excluding the start address and data length) of the Sync Write (0x83) command. The main interfaces are shown in the following table.

Method	Description
GroupSyncWrite	Constructor function
addParam	Add relevant parameters for the specified device ID
removeParam	Remove the relevant parameters of the specified device ID
changeParam	Modify the relevant parameters of the specified device ID
clearParam	Clear all device parameters
txPacket	Send a Sync Write (0x83) command with encapsulated parameters

5.3.1. GroupSyncWrite

GroupSyncWrite(port, ph, start_address, data_length)	
Functionality	Constructor function
Parameter	①port PortHandler instance ②ph Instance of Protocol1PacketHandler or Protocol2PacketHandler ③start_address Start address of the register to be written with data ④data_length Length of data that needs to be synchronously written for a single device
Return Value	None

5.3.2. addParam

addParam(siar_id, data)	
Functionality	Add relevant parameters for the specified device ID
Parameter	①siar_id Device ID ②data Data to be written
Return Value	Returns True

5.3.3. removeParam

removeParam(siar_id)	
Functionality	Remove the relevant parameters of the specified device ID
Parameter	①siar_id Device ID
Return Value	False: When the device ID list does not contain data for the specified device ID, the removal fails and returns False True: Successfully removed, returns True

5.3.4. changeParam

changeParam(siar_id, data)	
Functionality	Modify the relevant parameters of the specified device ID
Parameter	①siar_id Device ID ②data Update written data
Return Value	False: When the specified ID does not exist in the device ID list, the function will return false True: otherwise return true

5.3.5. clearParam

clearParam()	
Functionality	Clear all device parameters
Parameter	None
Return Value	None

5.3.6. txPacket

txPacket()	
Functionality	Send a Sync Write (0x83) command with encapsulated parameters
Parameter	None
Return Value	(result) result: communication result

5.4. GroupSyncRead

The GroupSyncRead class is responsible for encapsulating the parameters (excluding the start address and data length) of the Sync Read (0x82) command. The main interfaces are shown in the following table.

Method	Description
GroupSyncRead	Constructor function
addParam	Add the specified device ID to the device list
removeParam	Remove the specified device ID from the device list
clearParam	Clear Device List
txPacket	Send data packet
rxPacket	Receive data packets
txRxPacket	Send and receive data packets
isAvailable	Check if there is any available data among the read data
getData	Retrieve data from the received packet

5.4.1. GroupSyncRead

GroupSyncRead(port, ph, start_address, data_length)	
Functionality	Constructor function
Parameter	①port PortHandler instance ②ph Instance of Protocol1PacketHandler or Protocol2PacketHandler ③start_address Start address of the register for data to be read ④data_length Data Length
Return Value	None

5.4.2. addParam

addParam(siar_id)	
Functionality	Add the specified device ID to the device list
Parameter	①siar_id Device ID
Return Value	False: When using Protocol 1.0 or the specified device ID is already in the device list True: Successfully added

5.4.3. removeParam

removeParam(siar_id)	
Functionality	Remove the specified device ID from the device list
Parameter	①siar_id Device ID
Return Value	False: When using Protocol 1.0 or when the specified device ID does not exist in the device list True: Successfully removed

5.4.4. clearParam

clearParam()	
Functionality	Clear Device List
Parameter	None
Return Value	None

5.4.5. txPacket

txPacket()	
Functionality	Send data packet
Parameter	None
Return Value	(result) result: communication result

5.4.6. rxPacket

rxPacket()	
Functionality	Receive data packets
Parameter	None
Return Value	None

5.4.7. txRxPacket

rxPacket()	
Functionality	Send and receive data packets
Parameter	None
Return Value	(rkxpacet, result) rkxpacet: Received data packets result: communication result

5.4.8. isAvailable

isAvailable(siar_id, address, data_length)	
Functionality	Check if there is any available data among the read data
Parameter	①siar_id Device ID ②address Register Address ③ data_length Data Length
Return Value	False: No data available True: There is data of length data_length starting from the specified memory table address address

5.4.9. getData

getData(siar_id, address, data_length)	
Functionality	Retrieve data from the received packet
Parameter	①siar_id Device ID ②address Register Address ③ data_length Data Length
Return Value	0: No data was obtained Other: Read data value

5.5. GroupBulkWrite

The GroupBulkWrite class is responsible for encapsulating the parameters of the Bulk Write (0x93) command. The main interfaces are shown in the following table.

Method	Description
GroupSyncWrite	Constructor function
addParam	Add relevant parameters for the specified device ID
removeParam	Remove the relevant parameters of the specified device ID
changeParam	Modify the relevant parameters of the specified device ID
clearParam	Clear all device parameters
txPacket	Send a synchronous write Bulk Write (0x93) command with encapsulated parameters

5.5.1. groupBulkWrite

GroupBulkWrite(port, ph)	
Functionality	Constructor function
Parameter	①port PortHandler instance ②ph Instance of Protocol1PacketHandler or Protocol2PacketHandler
Return Value	None

5.5.2. addParam

addParam(siar_id, start_address, data_length, data)	
Functionality	Add relevant parameters for the specified device ID
Parameter	①siar_id Device ID ②start_address Register Address ③data_length Length of data to be written ④data Data to be written
Return Value	False: Addition failed Ture: Added successfully

5.5.3. removeParam

removeParam(siar_id)	
Functionality	Remove the relevant parameters of the specified device ID
Parameter	①siar_id Device ID
Return Value	False: When no device data is specified in the parameter device list, the function returns false, True: Successfully removed

5.5.4. changeParam

changeParam(siar_id, start_address, data_length, data)	
Functionality	Modify the relevant parameters of the specified device ID
Parameter	①siar_id Device ID ②start_address Starting address of the register to be written with data ③data_length Length of data to be written ④data Data to be written
Return Value	False: When the target ID does not exist in the ID list, the function will return false True: Successfully modified

5.5.5. clearParam

clearParam()	
Functionality	Clear all device parameters
Parameter	None
Return Value	None

5.5.6. txPacket

txPacket()	
Functionality	Send a synchronous write Bulk Write (0x93) command with encapsulated parameters
Parameter	None
Return Value	(result) result: communication result

5.6. GroupBulkRead

The GroupBulkRead class is responsible for encapsulating the parameters of the Bulk Write (0x92) command for batch reading. The main interfaces are shown in the following table.

Method	Description
GroupBulkRead	Constructor function
addParam	Add relevant parameters for the target device ID
removeParam	Remove relevant parameters of the target device ID
clearParam	Clear all device parameters
txPacket	Send data packet
rxPacket	Receive data packets
txRxPacket	Send and receive data packets
isAvailable	Check if there is any available data among the read data
getData	Retrieve data from the received packet

5.6.1. GroupBulkRead

GroupBulkRead(port, ph)	
Functionality	Constructor function
Parameter	①port PortHandler instance ②ph Instance of Protocol1PacketHandler or Protocol2PacketHandler
Return Value	None

5.6.2. addParam

addParam(siar_id, start_address, data_length)	
Functionality	Add relevant parameters for the target device ID
Parameter	①siar_id Device ID ②start_address Starting address of the register for data to be read ③data_length Data Length
Return Value	True: Added successfully False: Addition failed

5.6.3. removeParam

removeParam(siar_id)	
Functionality	Remove relevant parameters of the target device ID
Parameter	①siar_id Device ID
Return Value	None

5.6.4. clearParam

clearParam()	
Functionality	Clear all device parameters
Parameter	①siar_id Device ID
Return Value	None

5.6.5. txPacket

txPacket()	
Functionality	Send data packet
Parameter	None
Return Value	None

5.6.6. rxPacket

rxPacket()	
Functionality	Receive data packets
Parameter	None
Return Value	None

5.6.7. txRxPacket

txRxPacket()	
Functionality	Send and receive data packets
Parameter	None
Return Value	None

5.6.8. isAvailable

isAvailable(siar_id, address, data_length)	
Functionality	Check if there is any available data among the read data
Parameter	①siar_id Device ID ②address Register Address ③data_length Data Length
Return Value	False: Returns false when there is no data from the target address True: otherwise return true

5.6.9. getData

getData(siar_id, address, data_length)	
Functionality	Retrieve data from the received packet
Parameter	①siar_id Device ID ②address Register Address ③data_length Data Length
Return Value	False: Returns false when there is no data from the target address of the specified device True: otherwise return true